

ODSL Block Course

Lecture 11: Transformers

Nicole Hartman

nicole.hartman@tum.de

ODSL Block Course

26 Mar 2026



Our dreams for AI

Or my memory of them... circa 2017

Motivation: virtual assistant

Tell information



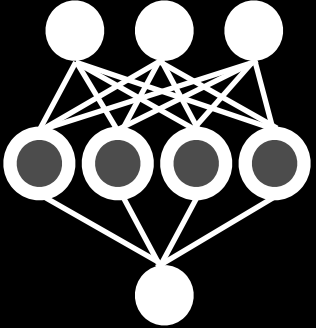
Ask questions



Use natural language!

Percy Liang, [ref](#)

Today... a dream come true?



Afternoon, Nicole

How can I help you today?



Opus 4.6 Extended ▾



 Code

 Learn

 Write

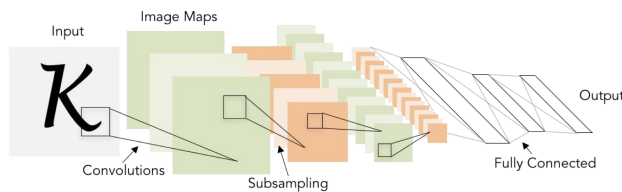
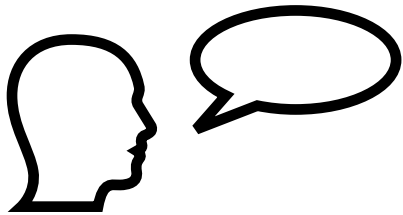
 Life stuff

 Claude's choice

2019: The bitter lesson (Richard Sutton)

“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin.”

Exs:



Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count

50,000,000,000

10,000,000,000

5,000,000,000

1,000,000,000

500,000,000

100,000,000

50,000,000

10,000,000

5,000,000

1,000,000

500,000

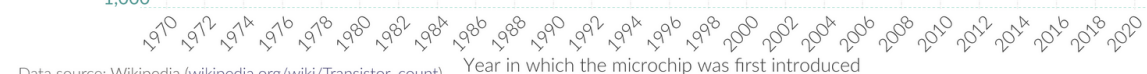
100,000

50,000

10,000

5,000

1,000



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

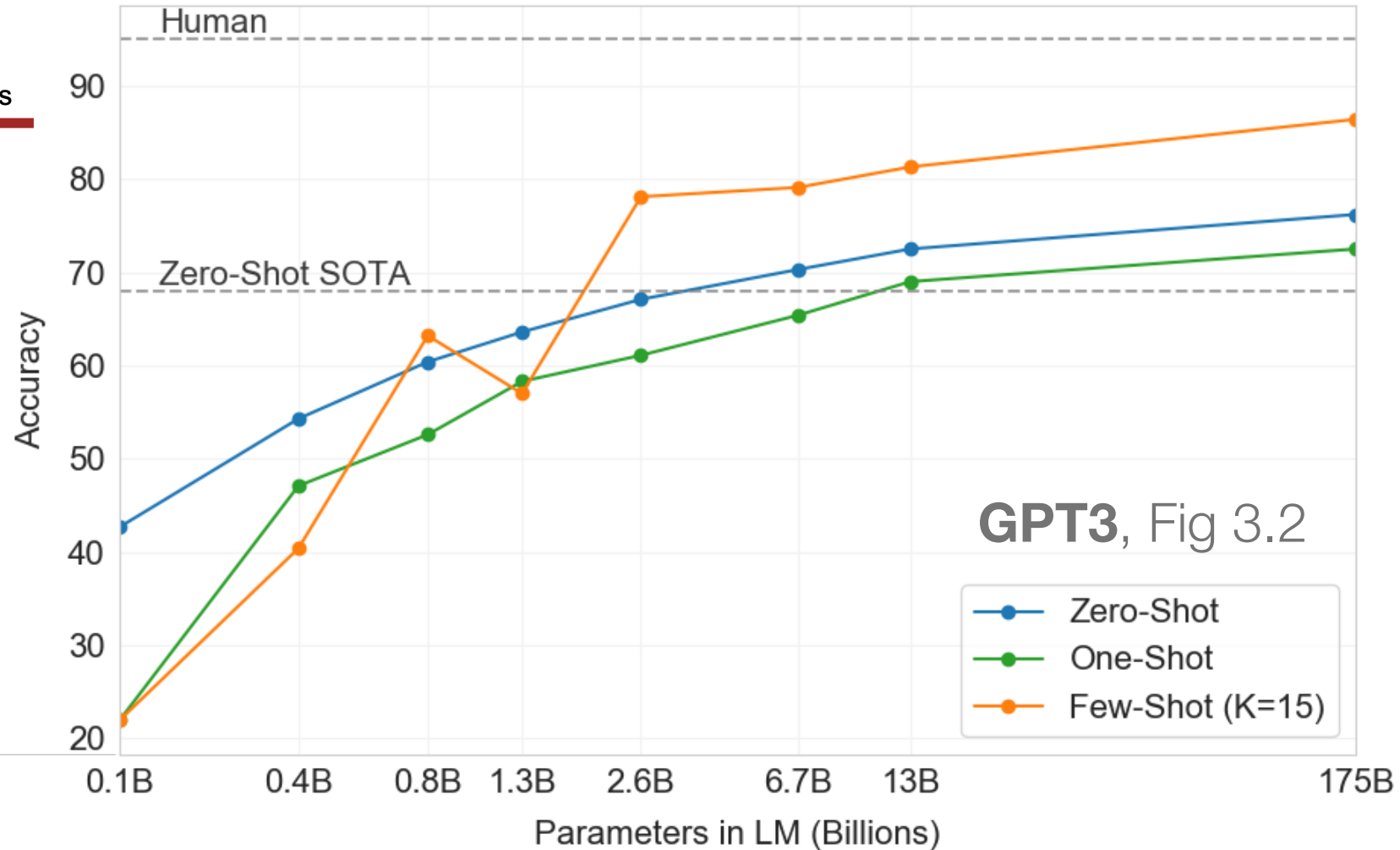
2020: Natural Language Processing



Comparing w/ size of
SOTA jet tagging models

✕ GN2 (2m) 2023

✕ GN3 (13m) 2025



Outline

- 1 How to represent the input
- 2 Training strategy
- 3 Transformers
- 4 The power of scale

Words -> integers



How you specify words in a dictionary for training language models

$$a = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

$$\text{aardvark} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

...

$$\text{exciting} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Specify the index in a set of classes

Index	Word
0	a
1	aardvark
$\vdots$	$\vdots$
i	Exciting
$\vdots$	$\vdots$
V	zoo

$\text{vocabulary_size} = V$

words in the dictionary

Characters -> integers



How you specify characters for training language models

$$a = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

$$b = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

...

$$z = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Specify the index in a set of classes

Index	Word
0	a
1	b
2	c
3	d
4	e
⋮	⋮
V-1	Z

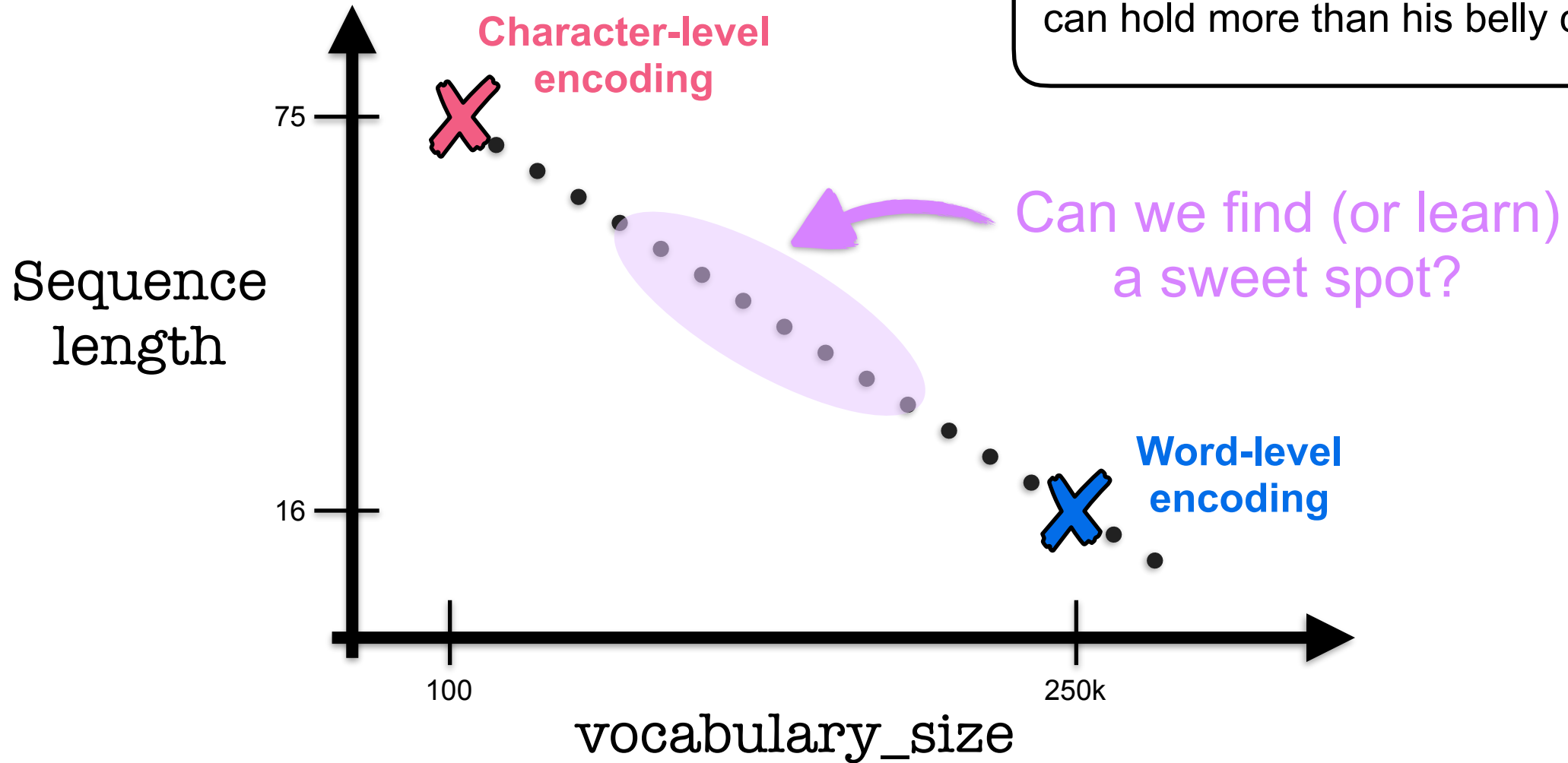
vocabulary_size = V

characters in language(s)

Input representation

Input sentence:

What a funny bird is a pellican, his beak
can hold more than his belly can.



Byte Pair Encoding (BPE)

Greedy algorithm to merge characters to make sub-words.

Initialize: split into characters

Start with individual characters (bytes). Count all adjacent pairs in the corpus.

l o w — l o w e r — n e w — l o w

Most frequent pair: ("l","o") appears 3 times

Byte Pair Encoding (BPE)

Greedy algorithm to merge characters to make sub-words.

Merge ("l","o") into "lo"

The most frequent pair becomes a new token. Re-count all pairs.

lo w — lo w e r — n e w — lo w

Most frequent pair: ("lo","w") appears 3 times

Merge rules learned so far:

l + o → lo

Byte Pair Encoding (BPE)

Greedy algorithm to merge characters to make sub-words.

Merge ("lo","w") into "low"

Pairs can include previously merged tokens. Vocabulary grows.

low low e r n e w low

Most frequent pair: ("low","e") appears 1 time (tie-break)

Merge rules learned so far:

l + o → lo lo + w → low

Byte Pair Encoding (BPE)

Greedy algorithm to merge characters to make sub-words.

Merge ("low","e") into "lowe"

Continue until desired vocabulary size is reached (e.g. 50k tokens).

low — lowe r — n e w — low

Next: keep merging the most frequent pair...

Merge rules learned so far:

l + o → lo lo + w → low low + e → lowe

Tiktokenizer

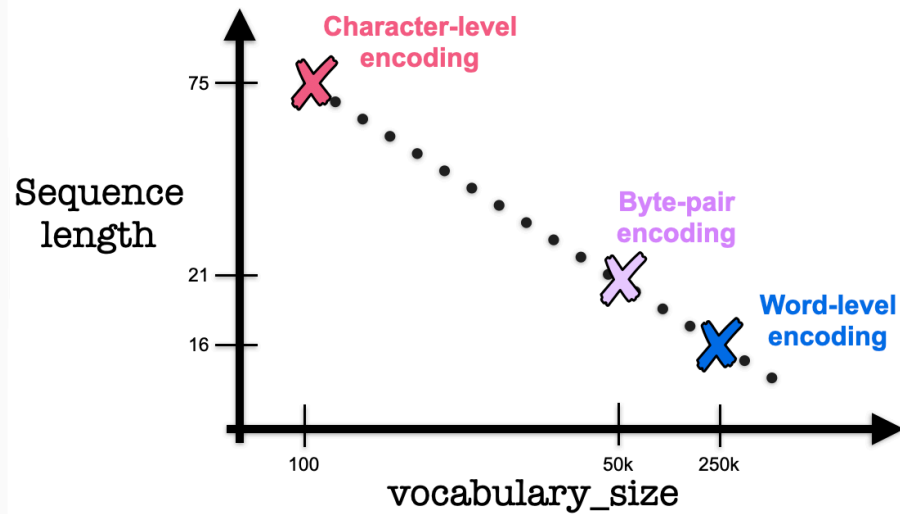
What a funny bird is a pellican, his beak can hold more than his belly can.

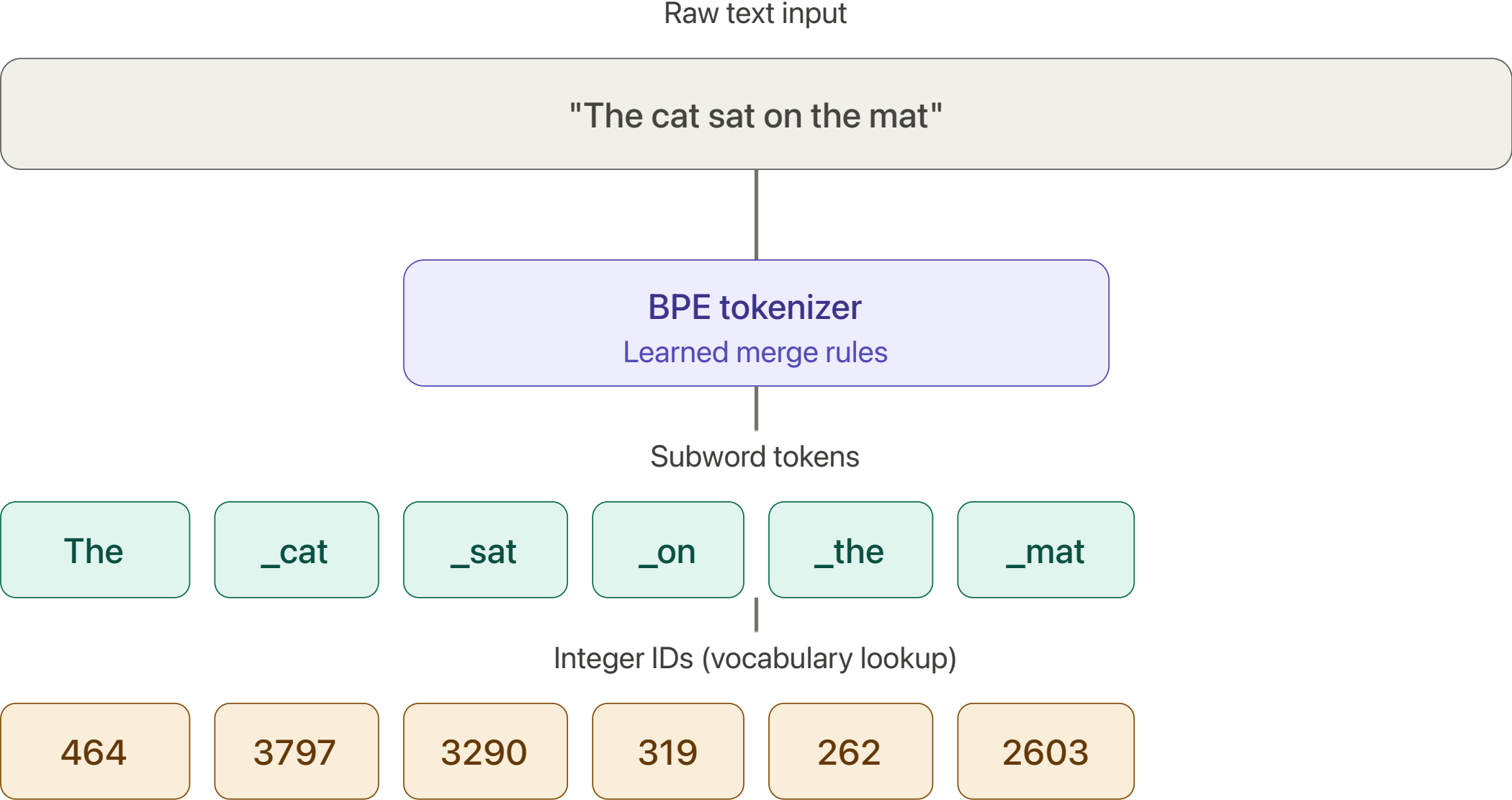
gpt2

Token count
21

What a funny bird is a pellican, his beak can hold more than his belly can.

2061, 257, 8258, 6512, 318, 257, 613, 297, 7490, 11, 465, 307, 461, 460, 1745, 517, 621, 465, 19921, 460, 13





Outline

1

How to represent the input

2

Training strategy

- Autoregressive model

3

Transformers

4

The power of scale

Autoregressive basics: chain rule

Bayes rule:

$$p(x, y) = p(x) \cdot p(y | x)$$

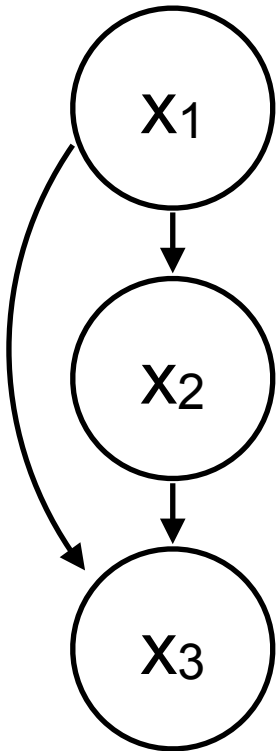
Chain rule: Any joint distribution can be written as a product of conditional distributions.

$$p(x_1, x_2, x_3) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2)$$

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$

Motivating example

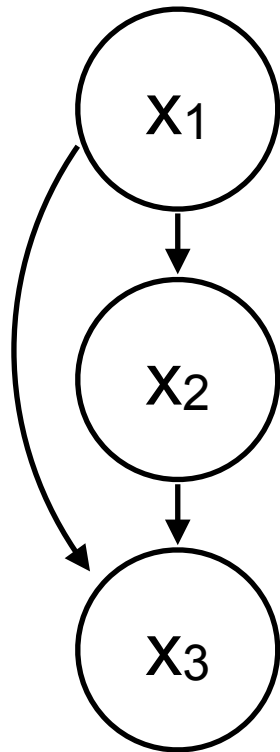
If we model the full probability distribution of words for a causally ordered sentence:



$$p(x_1, x_2, x_3) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2)$$

Motivating example

If we model the full probability distribution of words for a causally ordered sentence:



$$p(x_1, x_2, x_3) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2)$$

How

are

you

learnable parameters V

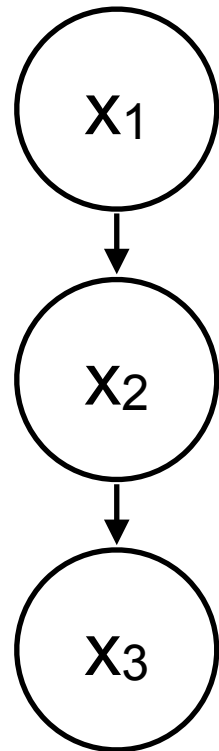
V^2

V^3

Motivating example

Simplifying assumption: $p(x_i | x_1, \dots, x_{i-1}) = p(x_i | x_{i-1})$

Pretend that only the previous word informs the prediction



$$p(x_1, x_2, x_3) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1)$$

How

are

you

learnable parameters V

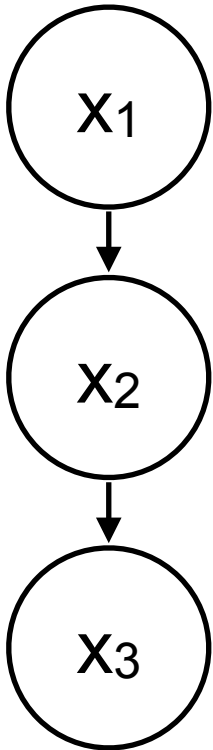
V^2

V^2

Motivating example

Simplifying assumption: $p(x_i | x_1, \dots, x_{i-1}) = p(x_i | x_{i-1})$

Pretend that only the previous word informs the prediction



$$p(x_1, x_2, x_3) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1)$$

How to train this distribution?

Looking for a distribution over a **set** of integers

Cross-entropy

	logits		Unnormalized probabilities		Class probabilities
	3.2		24.5		0.13
	5.1	$\xrightarrow{\text{exp}}$	164.0	$\xrightarrow{\text{normalize}}$	0.87
	-1.7		0.18		0.00

What type of loss function is this?

Cross-entropy loss

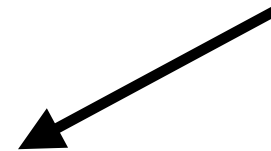
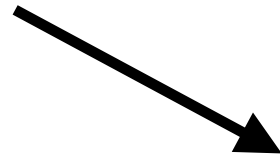
Given (set of) inputs, predict target

Supervised learning?

No “labels”

No “y” we just use the last element of x as a pseudo-label.... So really just learning $p(s)$

Unsupervised learning?



Self-supervised learning

Outline

1

How to represent the input

2

Training strategy

3

Transformers

- Attention
- Bells and whistles

4

The power of scale



Transformers 101

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

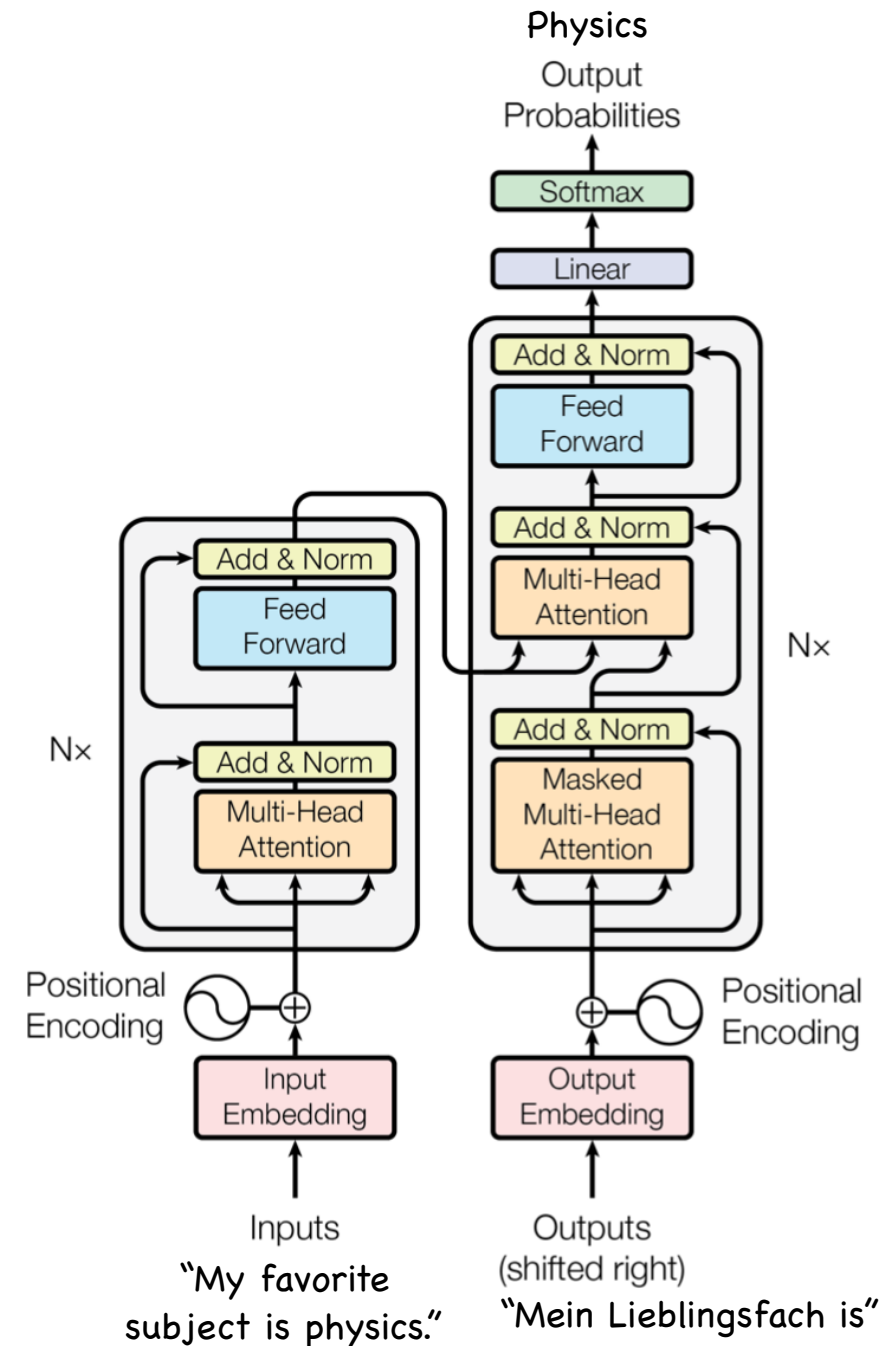
Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

200k
citations





Transformers 101

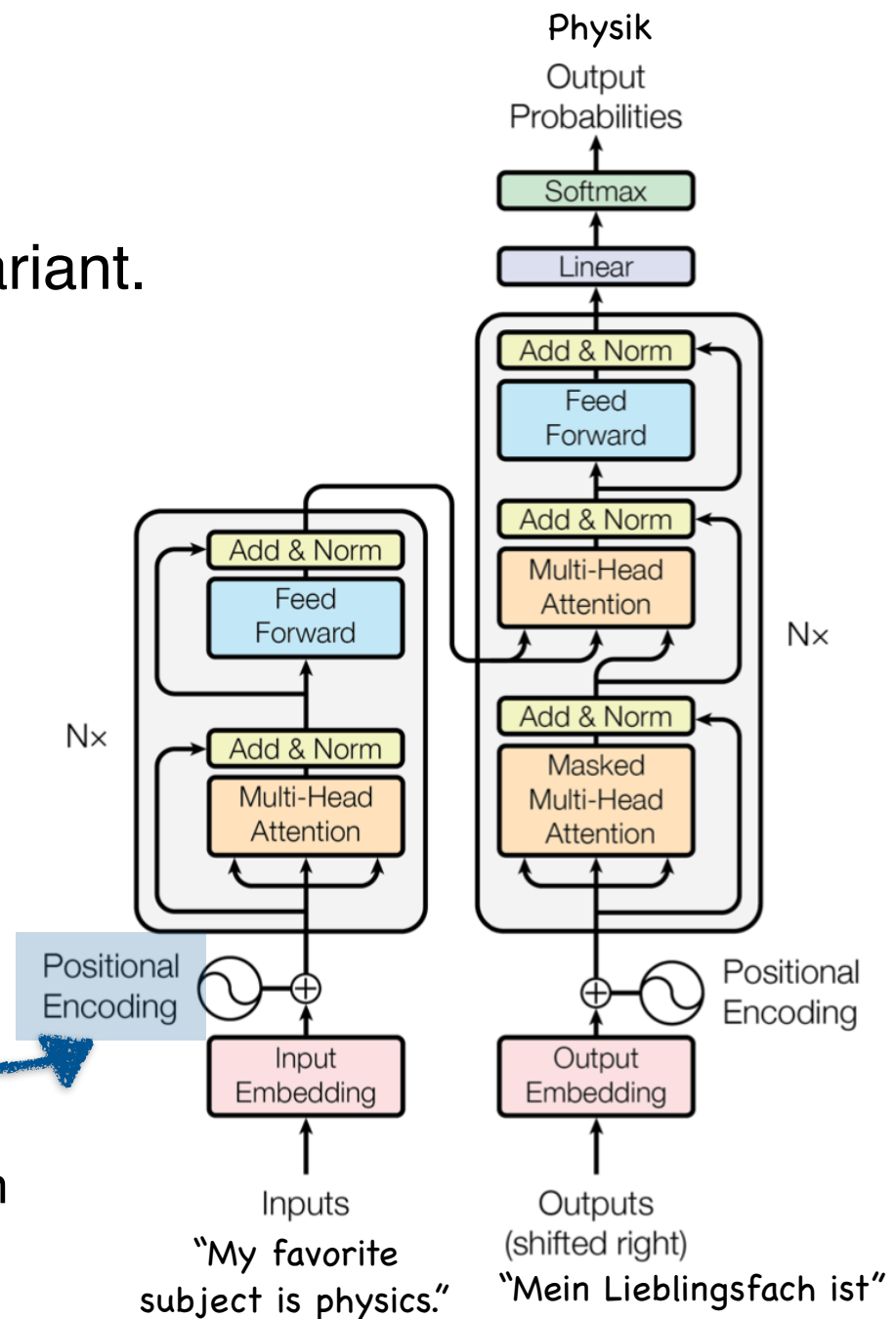
Architecture natively permutation invariant.



"My favorite subject is physics."

<START> 0 1 2 3 4 <END>

Sequence information encoded via an additional input





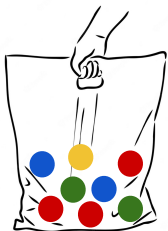
Transformers for FTAG

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$$

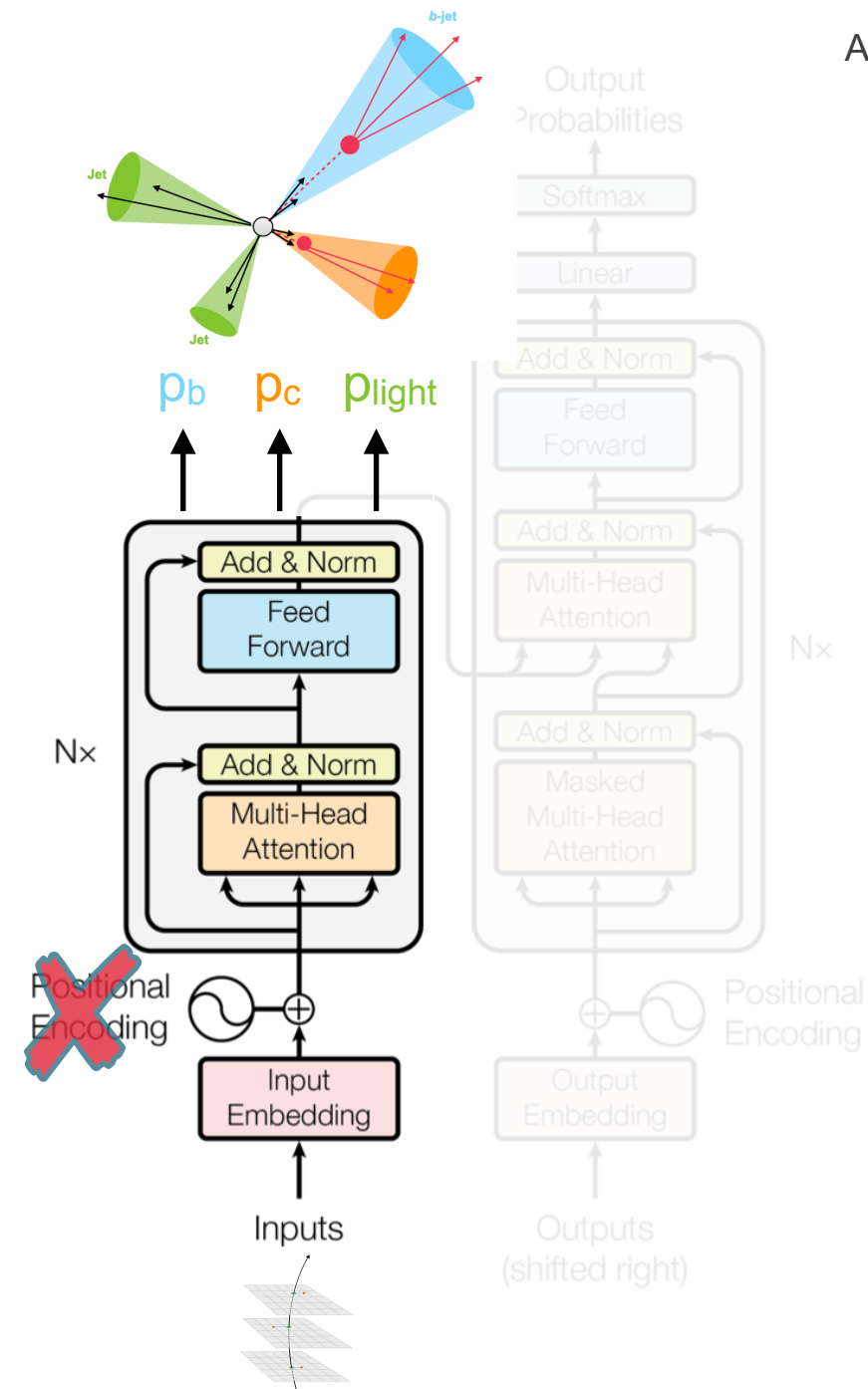
NLP: The **words** in a **sentence**

FTAG: **tracks** in a **jet**

Building up by analogy from our Deep Set from yesterday: look at the “encoder” piece of the transformer.

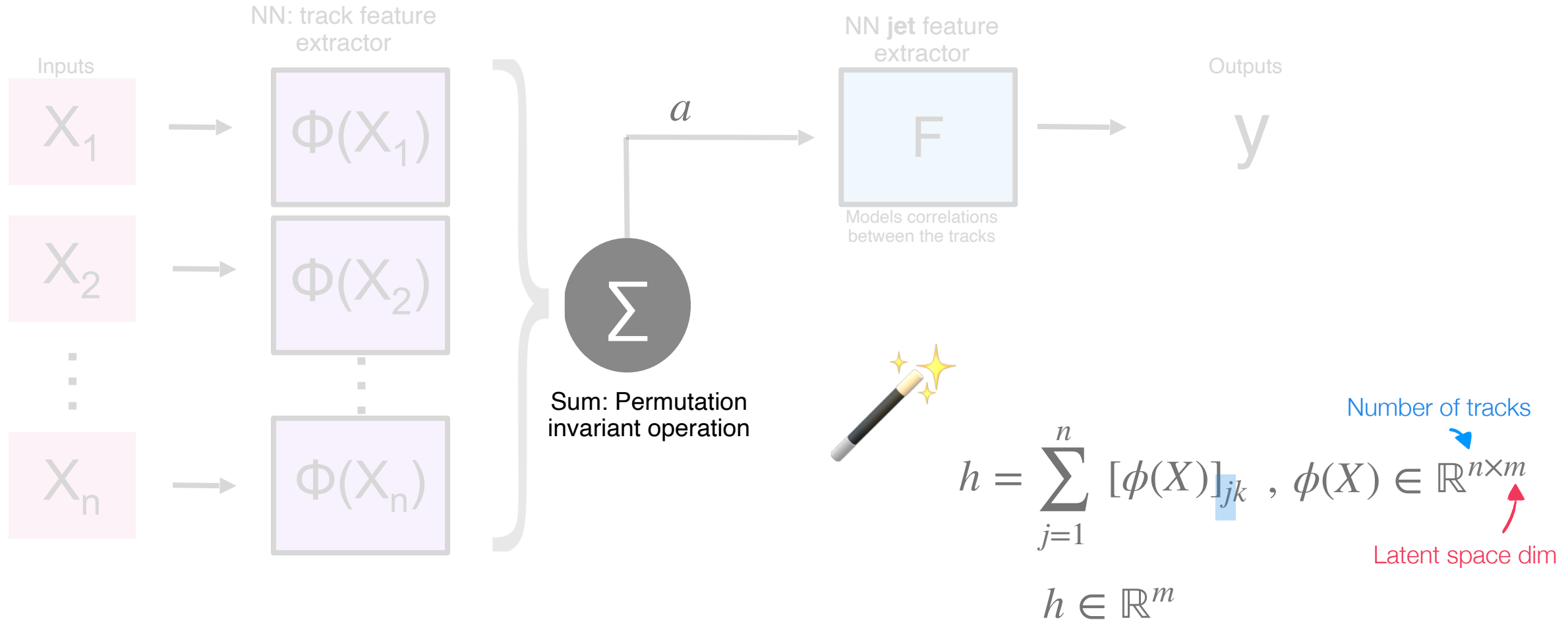


No need for positional encoding
(permutation invariance over the tracks)



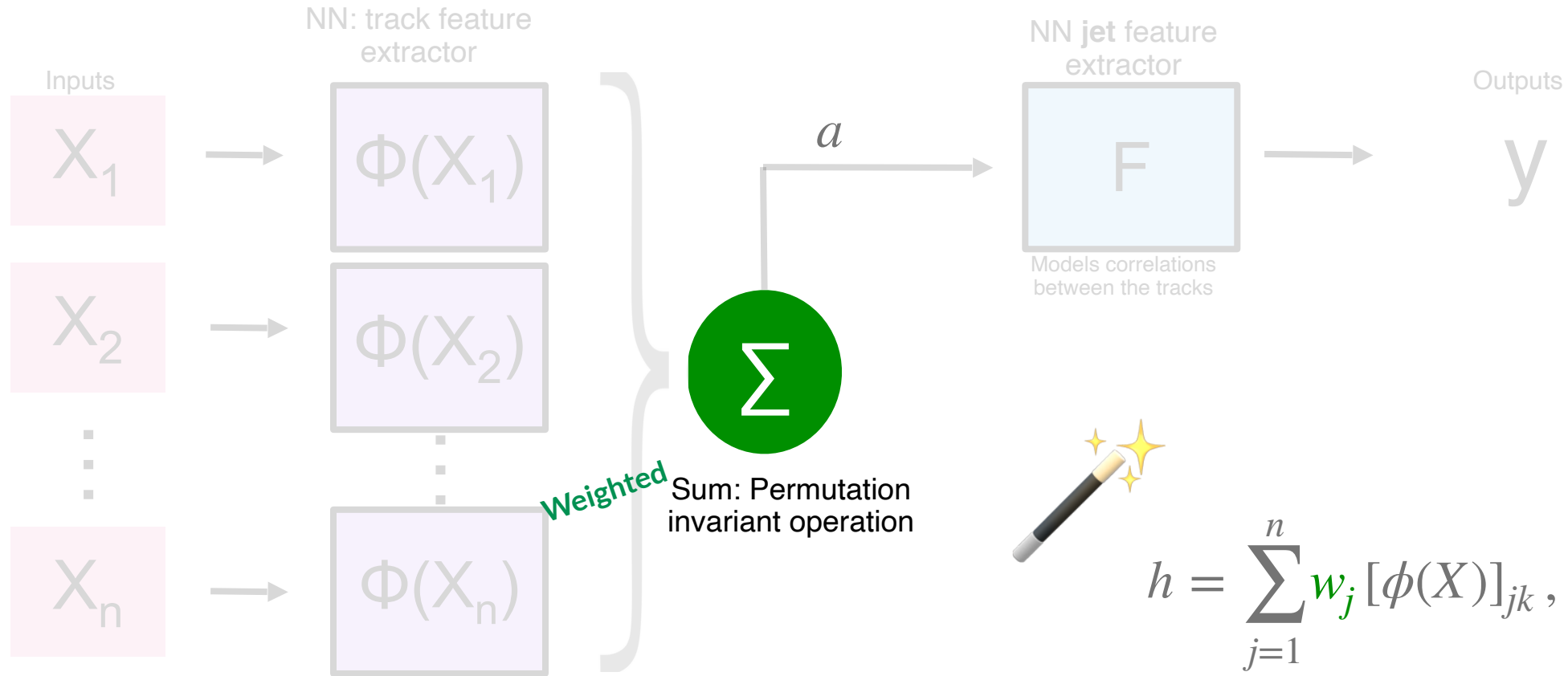
Recall (Deep Set): **permutation invariance**

A special type of function approximator



Also **permutation invariant**

Building up to transformer

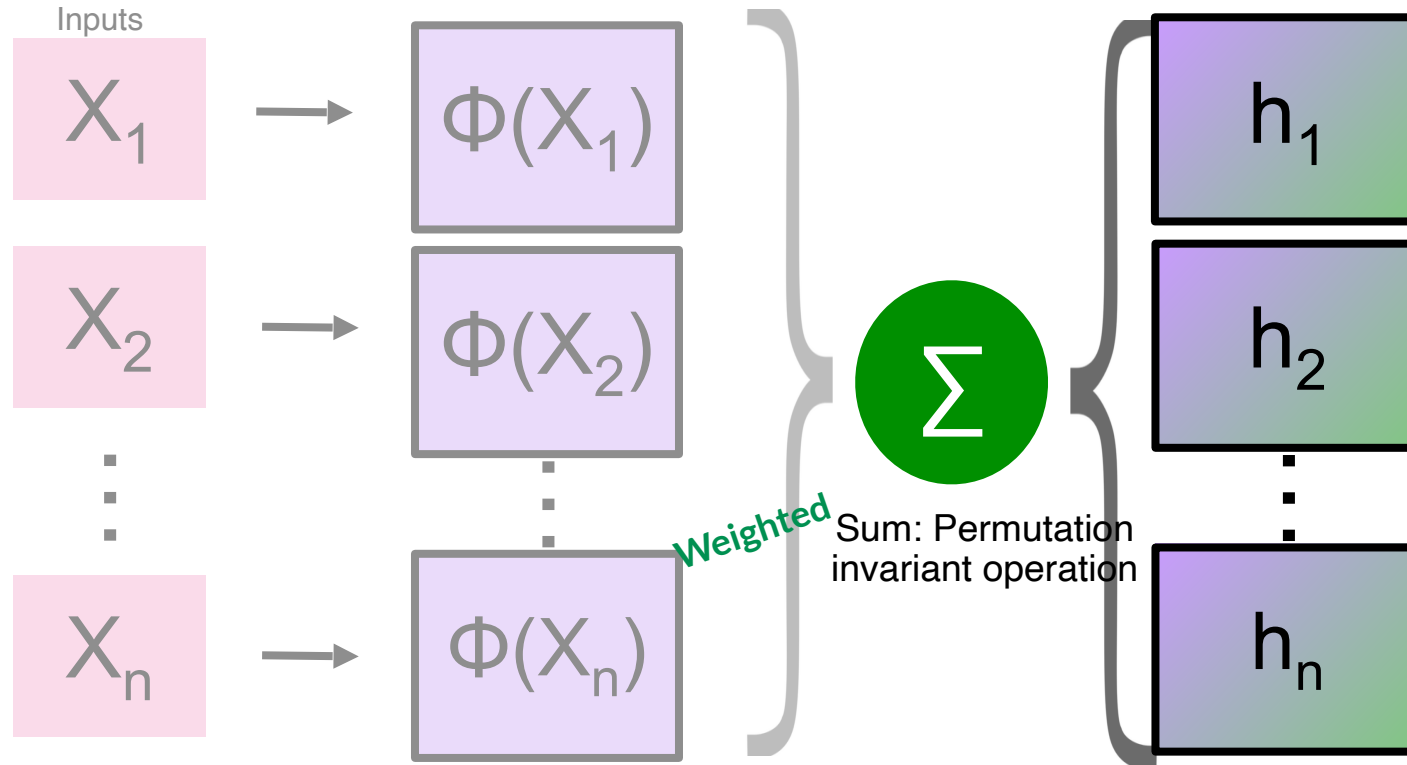


$$h = \sum_{j=1}^n w_j [\phi(X)]_{jk}, \phi(X) \in \mathbb{R}^{n \times m}$$

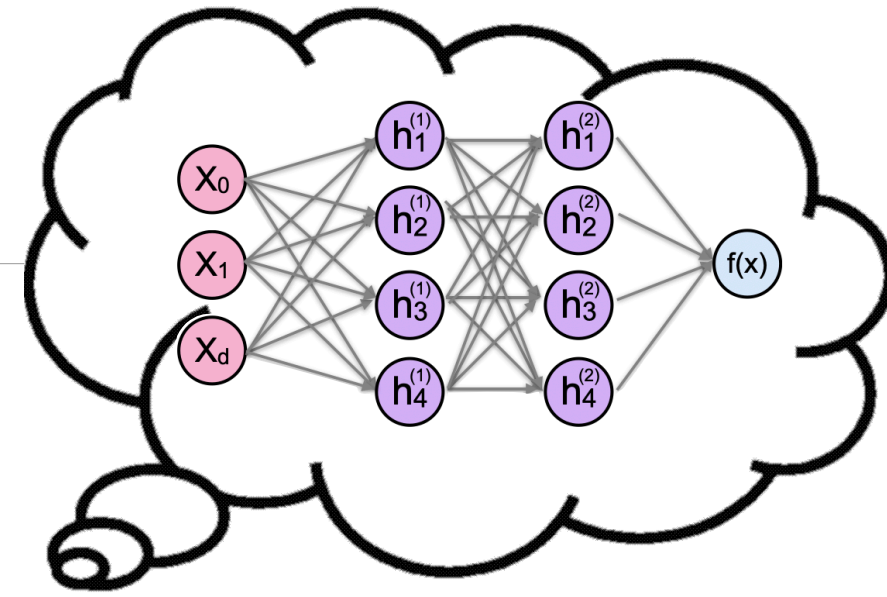
$$h \in \mathbb{R}^m$$

Building up (latent) complexity

Instead of extracting latent representation for nodes once (e.g, one application of ϕ)



Let the track latent be updated based on the context of the other tracks.

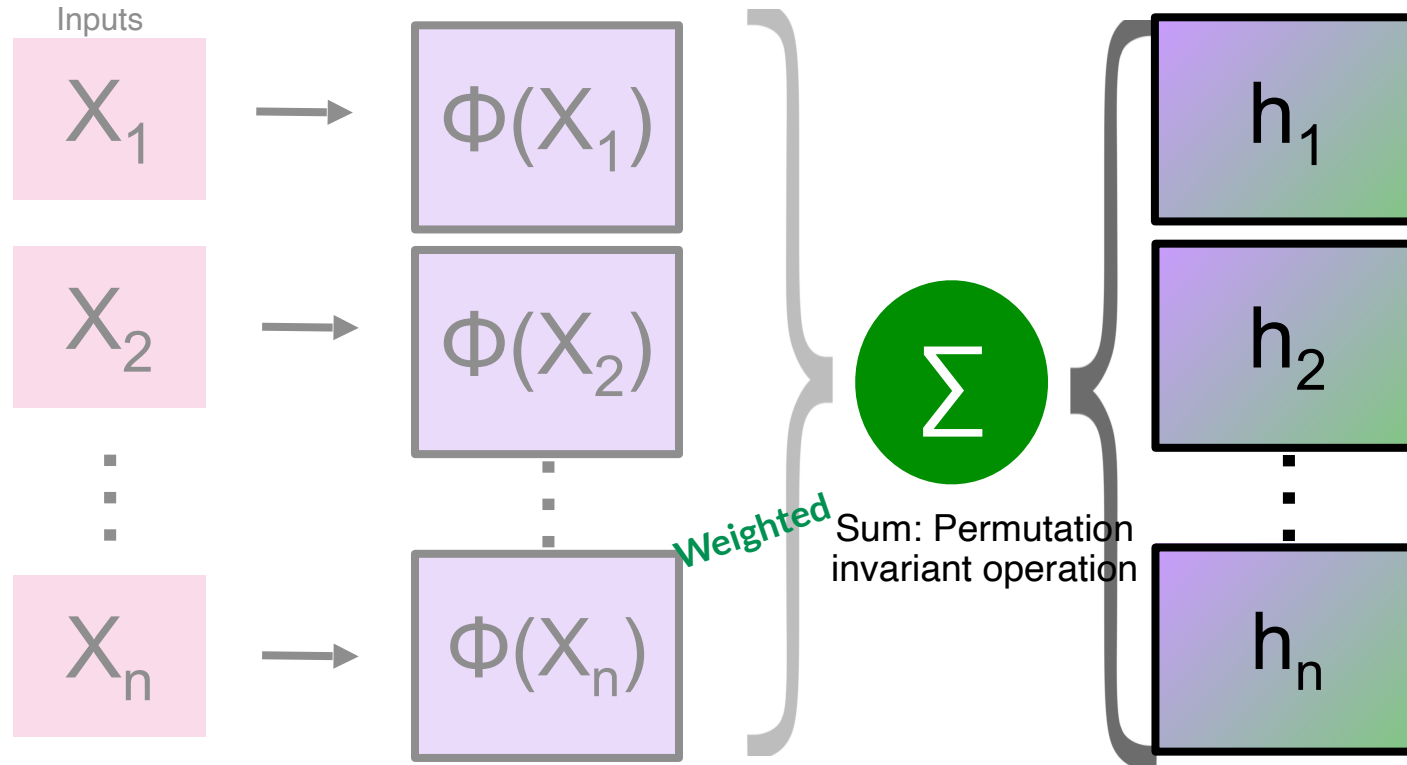


$$h = \sum_{j=1}^n w_j [\phi(X)]_{jk}, \phi(X) \in \mathbb{R}^{n \times m}$$

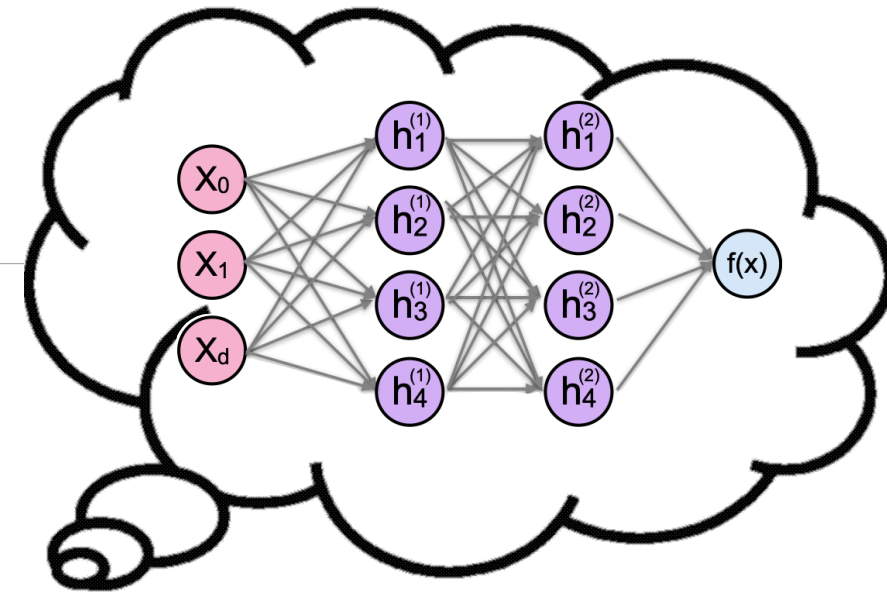
$$h \in \mathbb{R}^m$$

Also **permutation invariant**

Instead of extracting latent representation for nodes once (e.g, one application of ϕ)



Let the track latent be updated based on the context of the other tracks.



How to find these

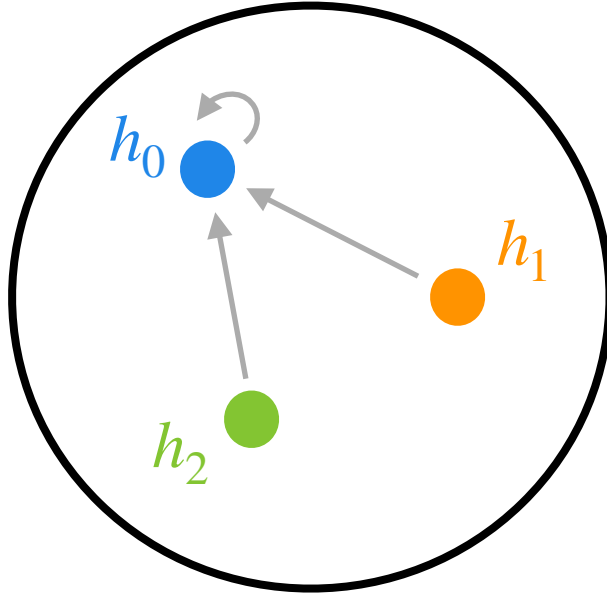
W_{ij} ?

$$h = \sum_{j=1}^n W_{ij} [\phi(X)]_{jk}, \phi(X) \in \mathbb{R}^{n \times m}$$

$$W_{ij} \in \mathbb{R}^{n \times n} \quad h \in \mathbb{R}^{n \times m}$$

Number of tracks

(Self)-Attention



Let h_i denote the hidden representation

E.g, $h_i = \phi(X_i) \in \mathbb{R}^m$, $m = \text{embed_dim}$

Tensor of all tracks (or words), $h \in \mathbb{R}^{n \times m}$

How can we update h_0 given the other nodes?

And... can we learn these updates by
gradient descent

(Self)-Attention

Want to find
these W_{ij}
track compatibility score

Tensor of all tracks, $h \in \mathbb{R}^{n \times m}$



Let's build a differentiable lookup table!

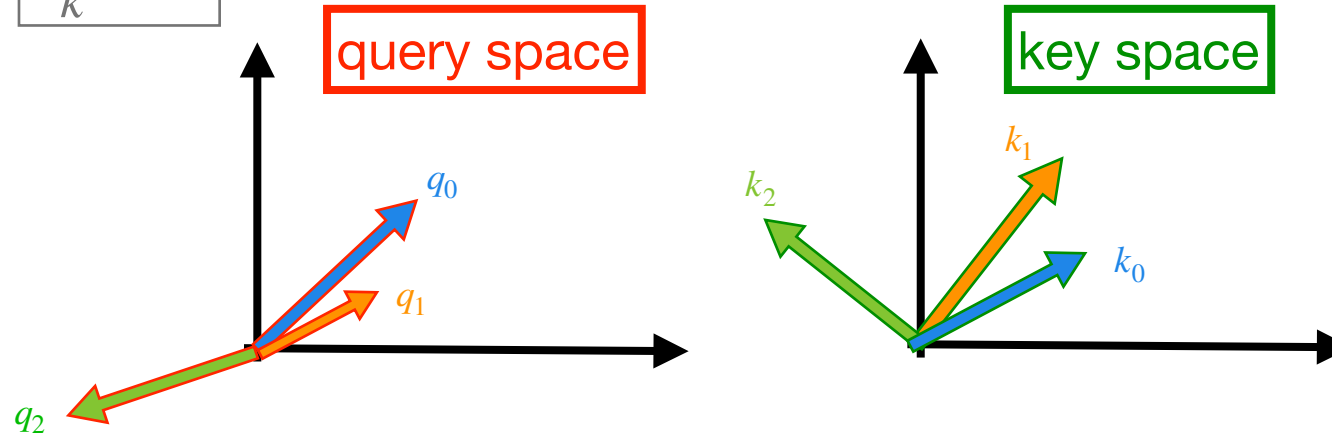
$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

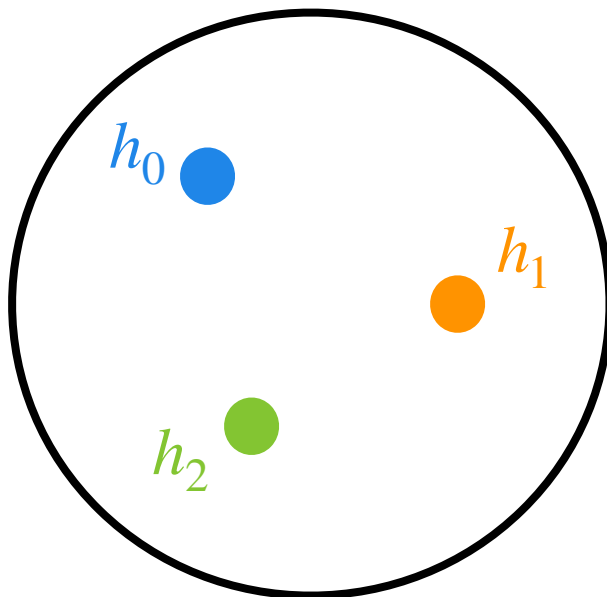
E.g,

$$d_k = 2$$



(Self)-Attention

Want to find
these W_{ij}
track compatibility score

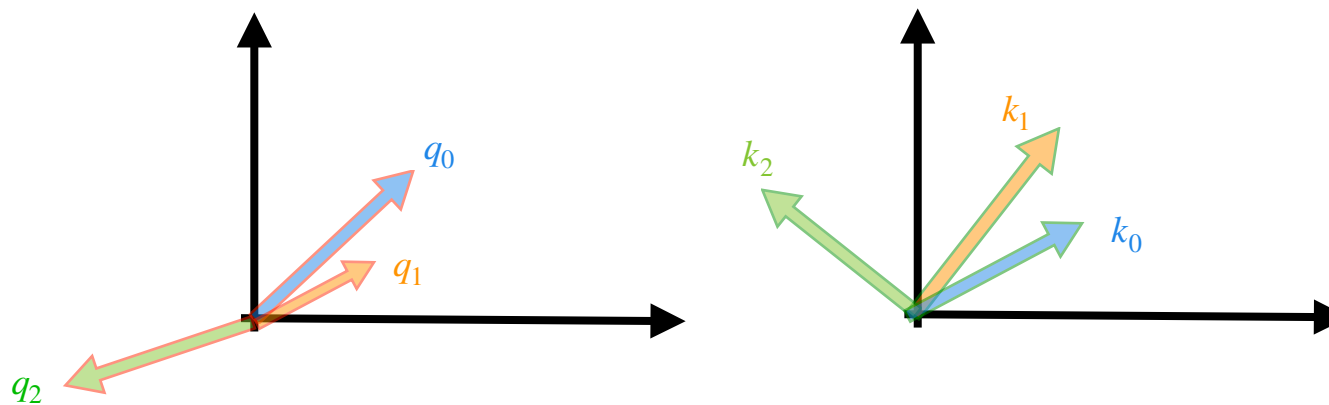


$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

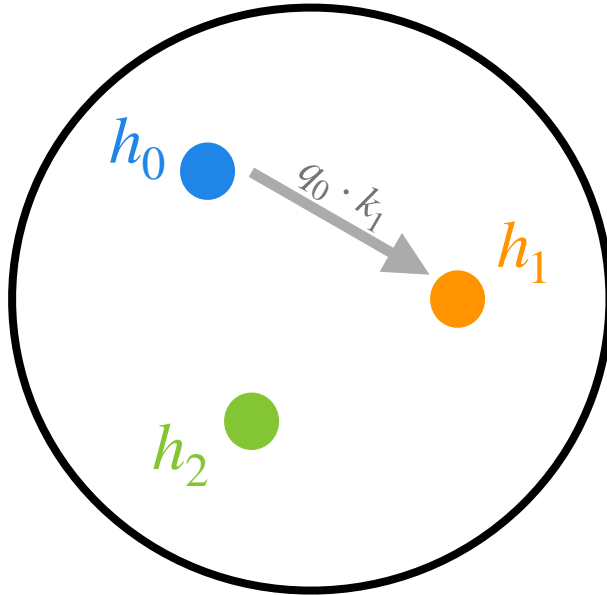
key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

Dot product of two vectors
measures the track compatibility



(Self)-Attention

Want to find
these W_{ij}
track compatibility score

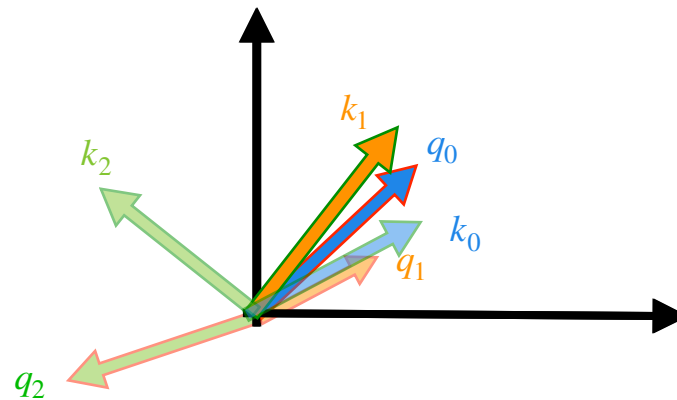


$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

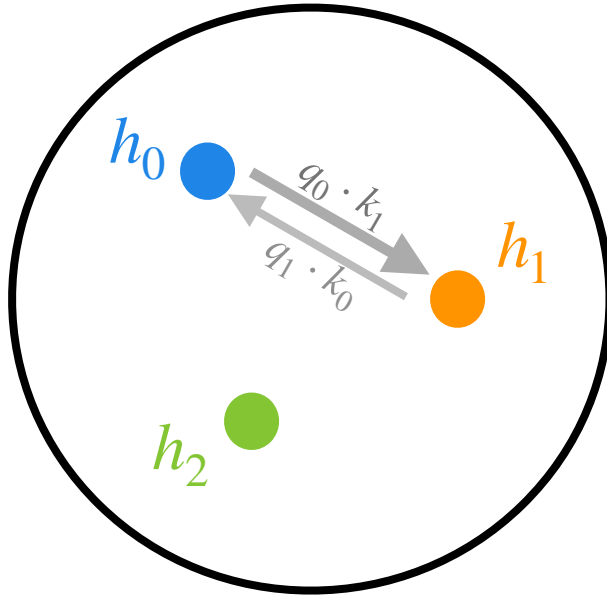
key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

Dot product of two vectors
measures the track compatibility



(Self)-Attention

Want to find
these W_{ij}
track compatibility score

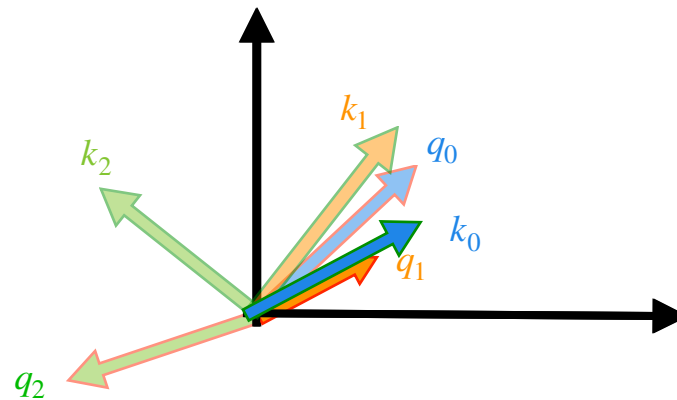


$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

Dot product of two vectors
measures the track compatibility

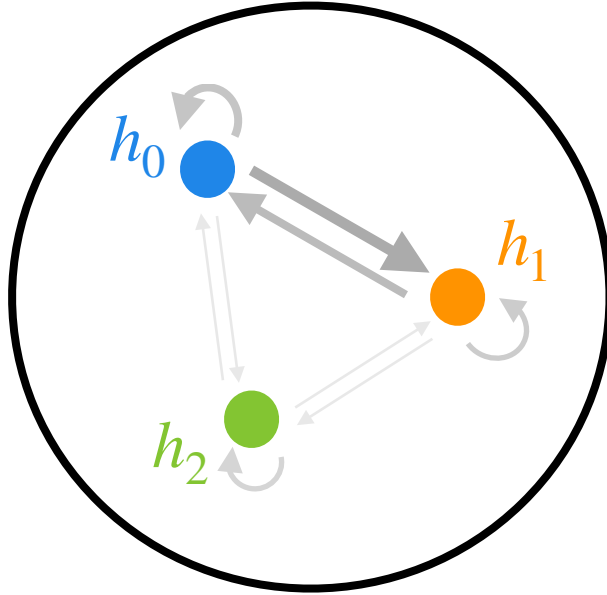


$Z = \begin{pmatrix} & q_0^T k_1 & \vdots \\ q_1^T k_0 & & \vdots \\ & & \ddots \end{pmatrix} \in \mathbb{R}^{n \times n}$

Logits
(unnorm prob)

Concept question

Want to find
these W_{ij}
track compatibility score



$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

Dot product of two vectors
measures the track compatibility

Q: How to write Z more succinctly?

B) qk^T

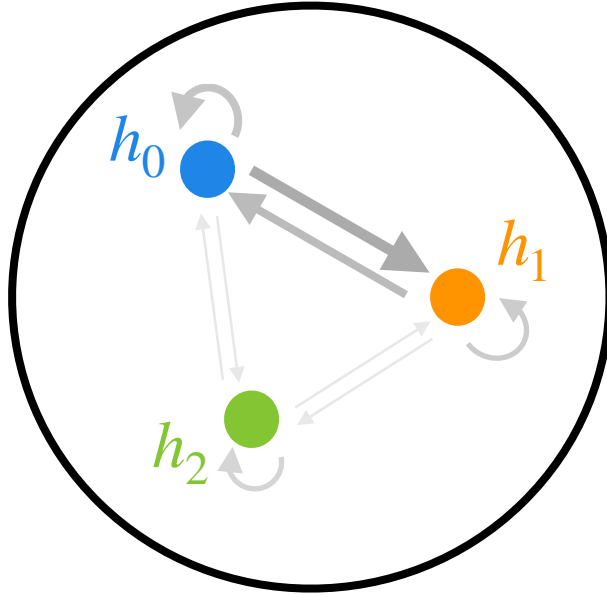
C) hh^T

$$Z = \begin{pmatrix} q_0^T k_0 & q_0^T k_1 & q_0^T k_2 \\ q_1^T k_0 & q_1^T k_1 & q_1^T k_2 \\ q_2^T k_0 & q_2^T k_1 & q_2^T k_2 \end{pmatrix} \in \mathbb{R}^{n \times n}$$



Concept question

Want to find
these W_{ij}
track compatibility score



$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

Dot product of two vectors
measures the track compatibility



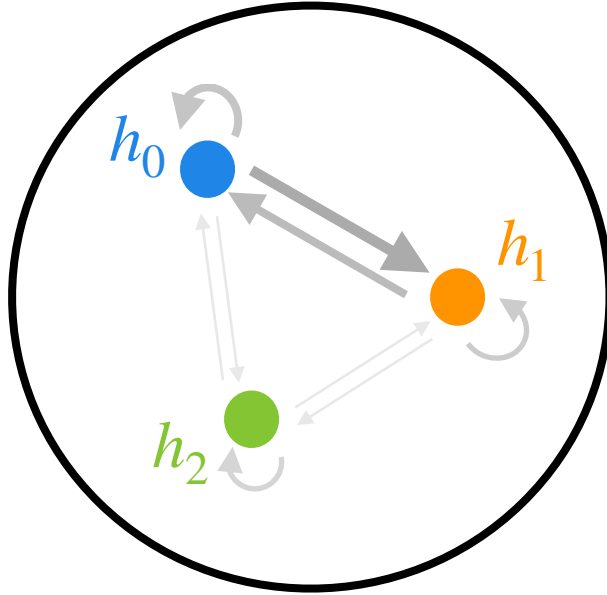
- A) $q^T k$
- B) qk^T
- C) hh^T

$$Z = \begin{pmatrix} q_0^T k_0 & q_0^T k_1 & q_0^T k_2 \\ q_1^T k_0 & q_1^T k_1 & q_1^T k_2 \\ q_2^T k_0 & q_2^T k_1 & q_2^T k_2 \end{pmatrix} \in \mathbb{R}^{n \times n}$$

Q: How to write Z more succinctly?

Self Attention

Want to find
these W_{ij}
track / word compatibility
score



$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

From yesterday, how to convert
logits into probs

Dot product of two vectors
measures the track compatibility

$$Z = qk^T \in \mathbb{R}^{n \times n}$$

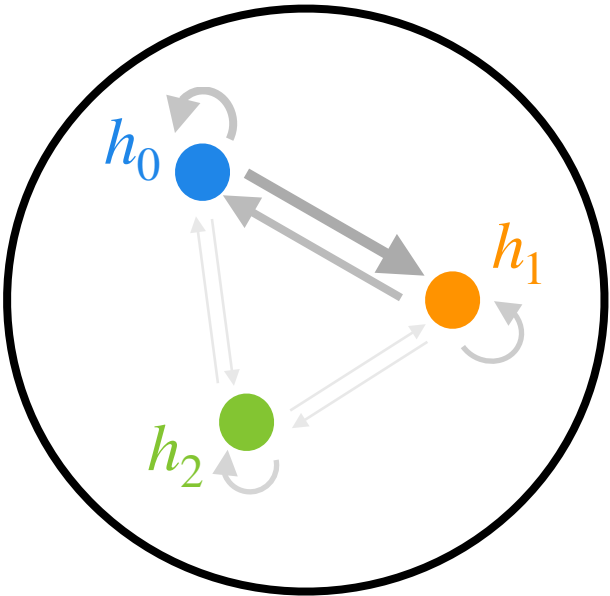
$$W = \text{Softmax} \left(\frac{qk^T}{\sqrt{d_k}} \right)$$

Nb: Softmax just normalizes
the rows of the matrix.

Norm by the $\sqrt{d_k}$

Self Attention (single head)

Want to find ~~these~~ **Found** these W_{ij}
track compatibility score



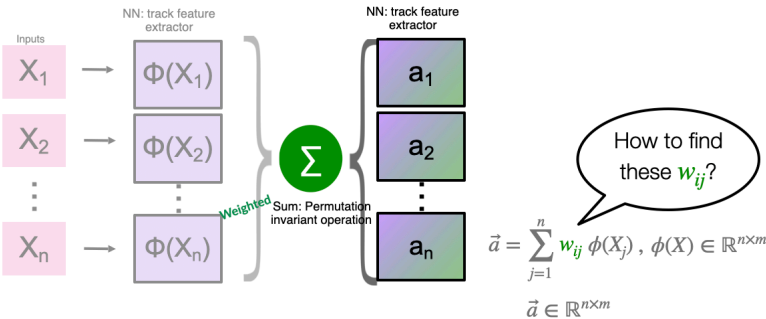
$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

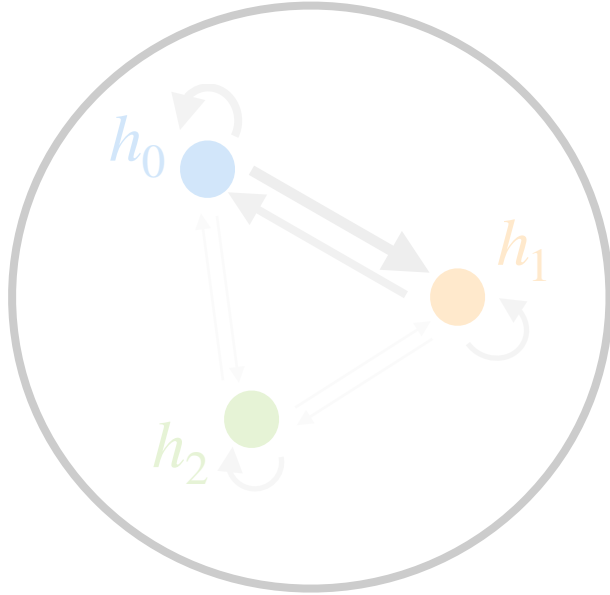
value: $v = hW_V \in \mathbb{R}^{n \times m}$ where $W_V \in \mathbb{R}^{m \times m}$

Can use these weights to update node representations



$$h \leftarrow \text{Softmax} \left(\frac{qk^T}{\sqrt{d_k}} \right) v$$

Issue: single head attention



$$W_Q, W_K \in \mathbb{R}^{m \times d_k}$$

query: $q = hW_Q$

key: $k = hW_K$

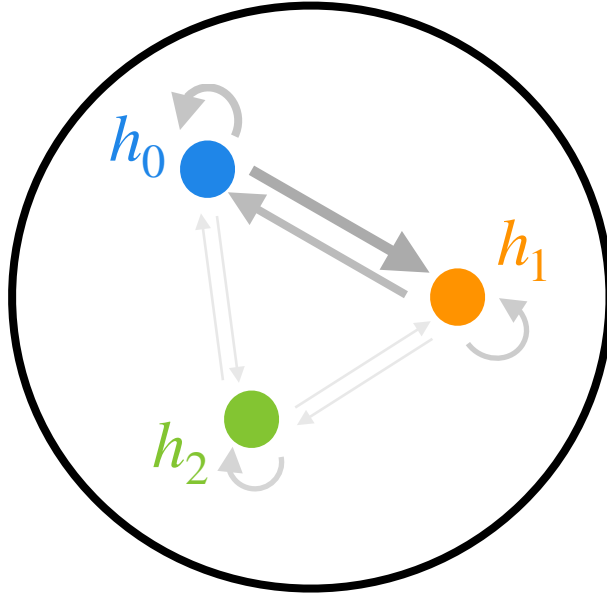
value: $v = hW_V$

Issue: for $m \gg d_k$, the
parameters of W_V dominate the
param count

Can use these weights to update node representations

$$h \leftarrow \text{Softmax} \left(\frac{qk^T}{\sqrt{d_k}} \right) v$$

Solution: Multihead attention



$W_Q, W_K \in \mathbb{R}^{m \times d_k}$: projection matrices

query: $q = hW_Q \in \mathbb{R}^{n \times d_k}$

key: $k = hW_K \in \mathbb{R}^{n \times d_k}$

value: $v = hW_V \in \mathbb{R}^{n \times d_v}$

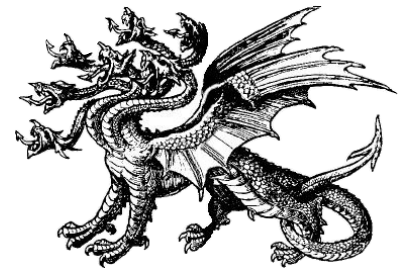
where $d_v = m/h$,

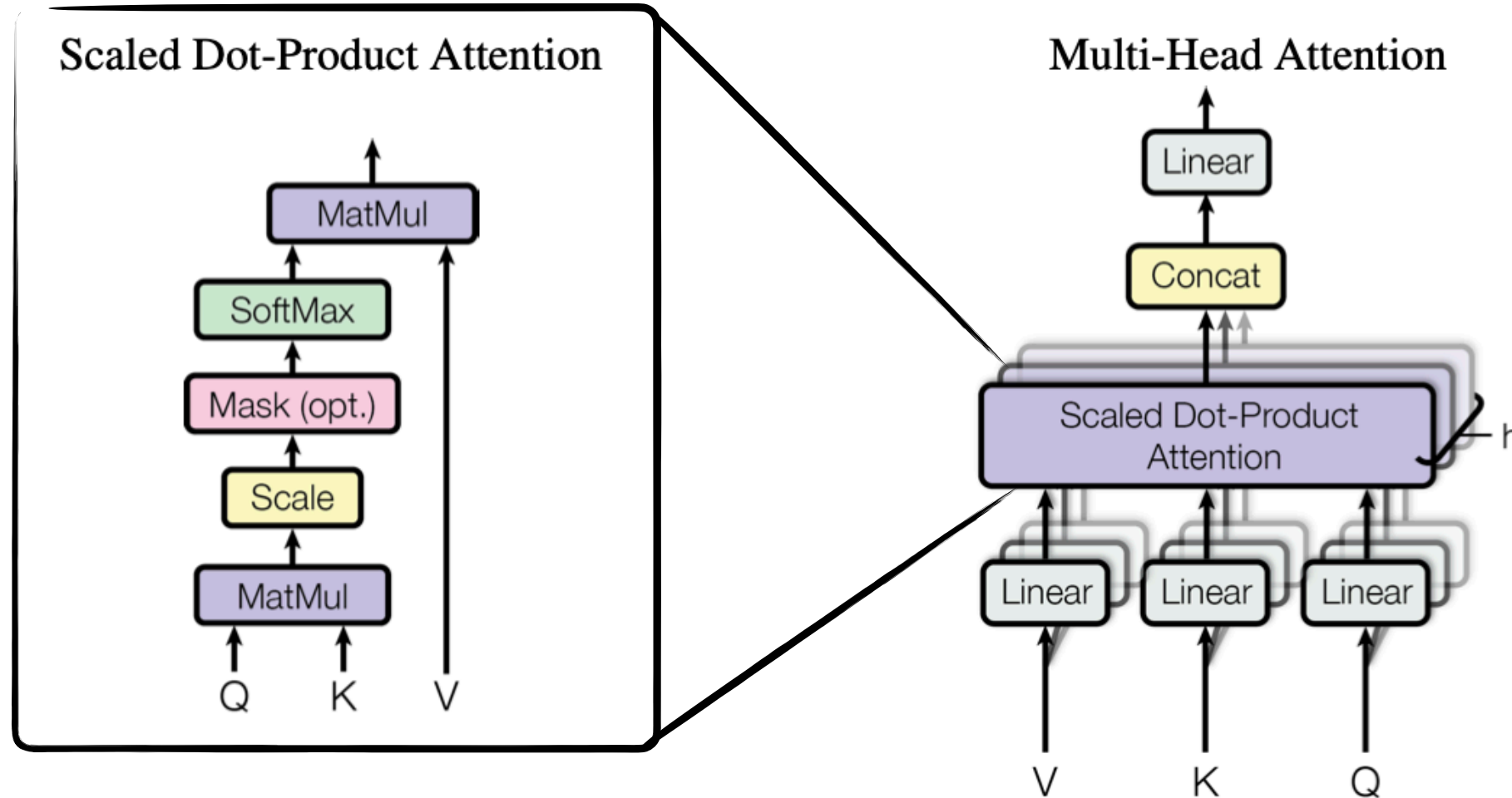
h = number of heads

(1) Partition this matrix

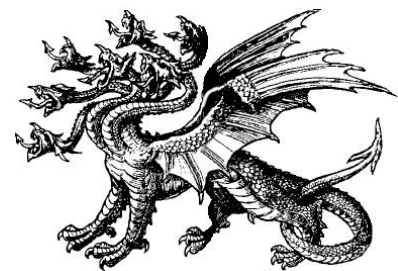
$W_V \in \mathbb{R}^{m \times d_v}$

(2) Repeat h times



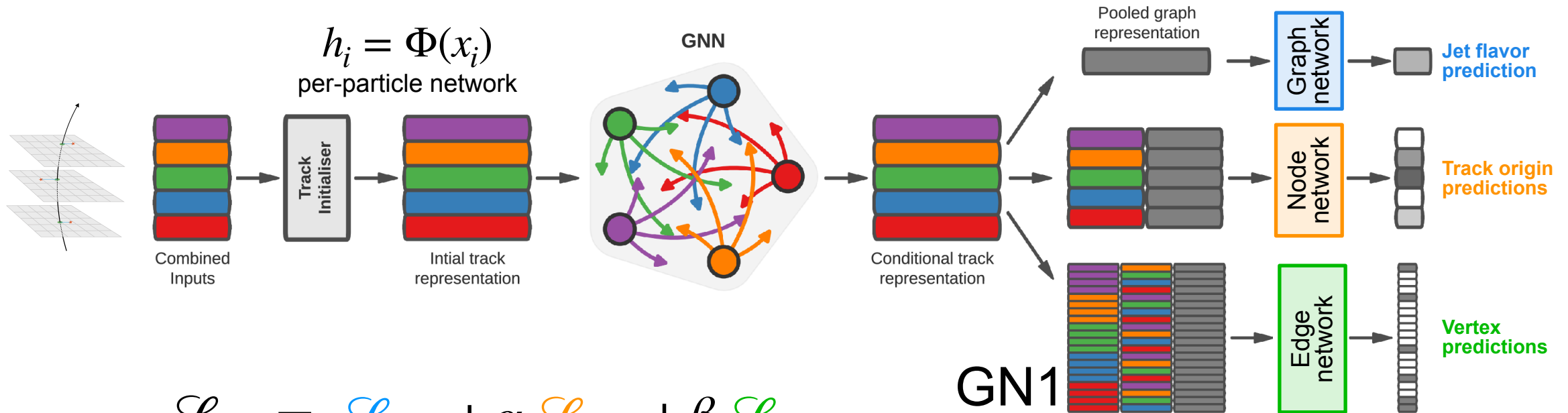


So... was attention the main trick of transformers?



Graph Neural Networks

And later... transformer-based



$$\mathcal{L}_{tot} = \mathcal{L}_{jet} + \alpha \mathcal{L}_{trk} + \beta \mathcal{L}_{vtx}$$

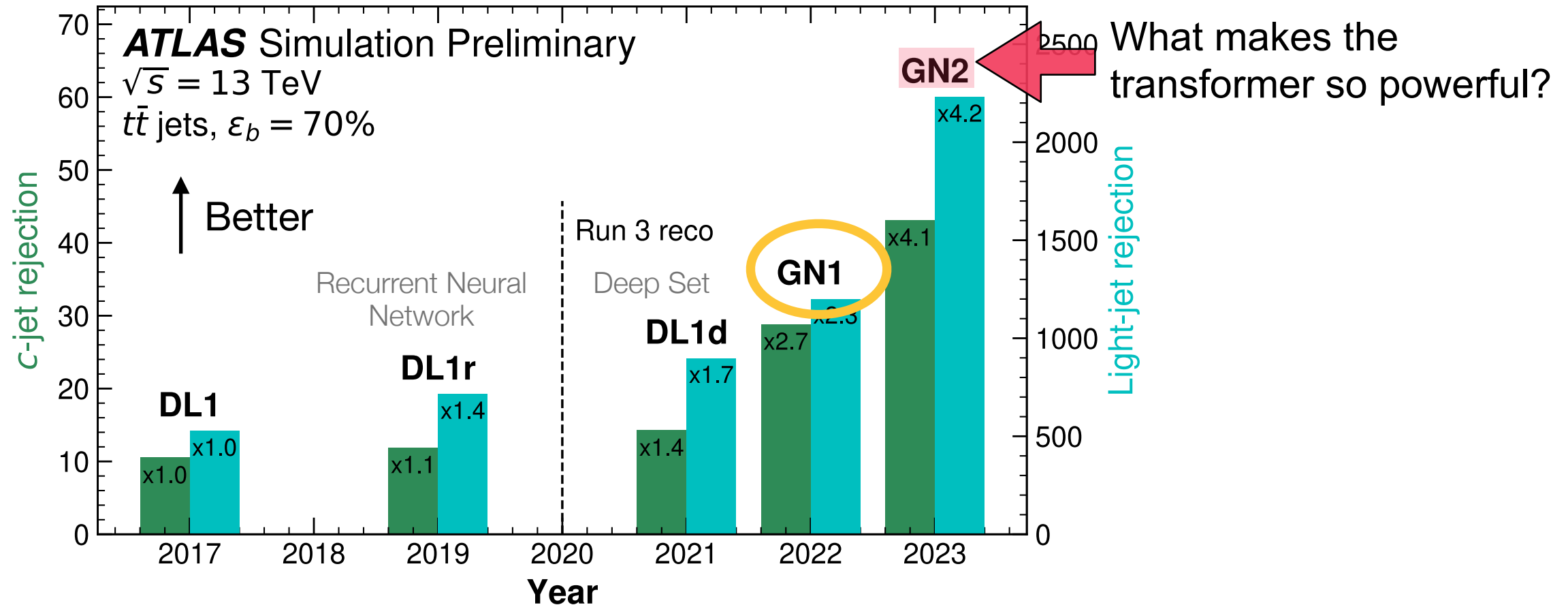


GN1: graph neural network

GN2: transformer

GN1
 already
 using
 multi-
 head
 attention
 (2 heads)

Attention (weighted sums) already used in Graph Neural Networks





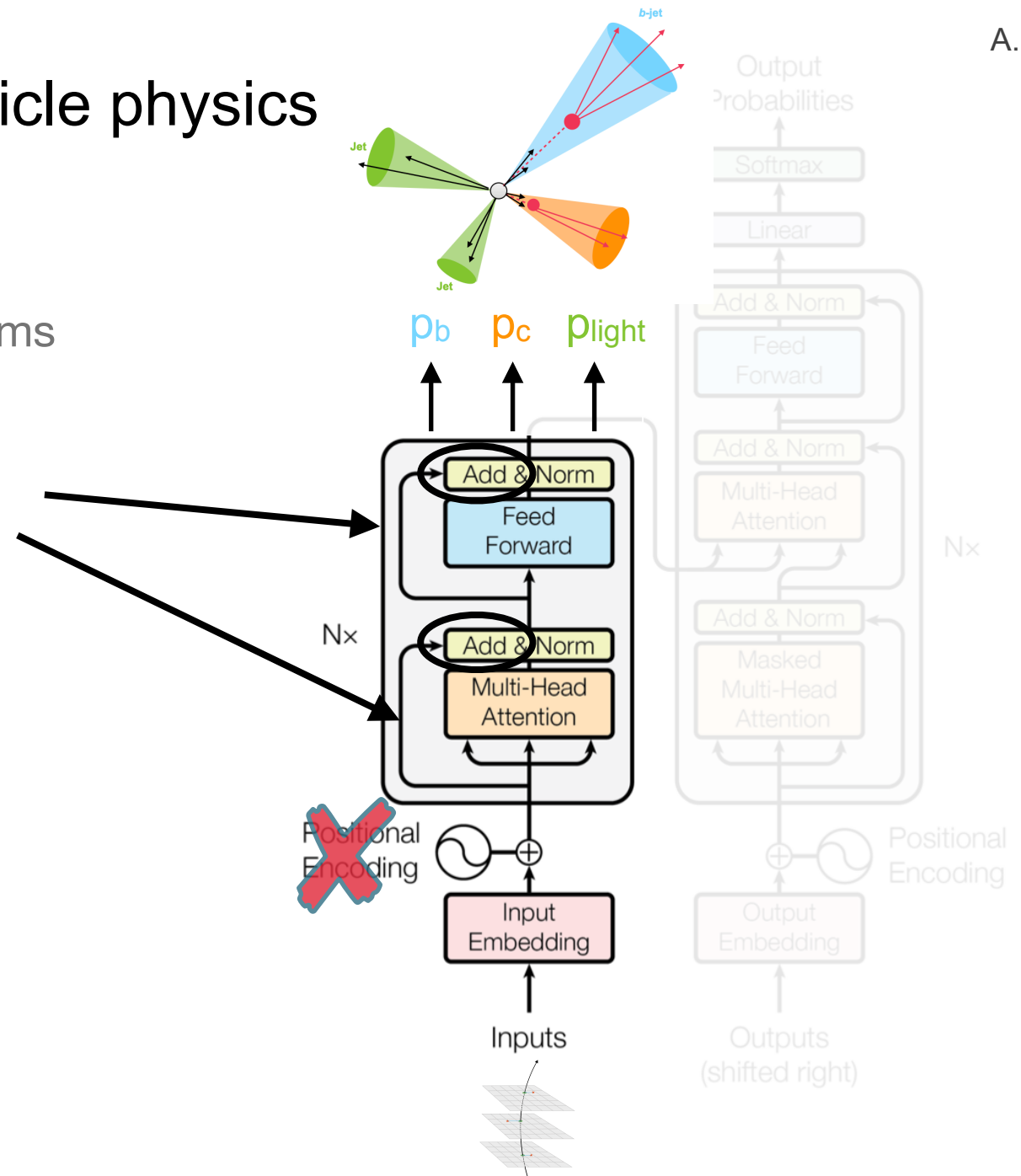
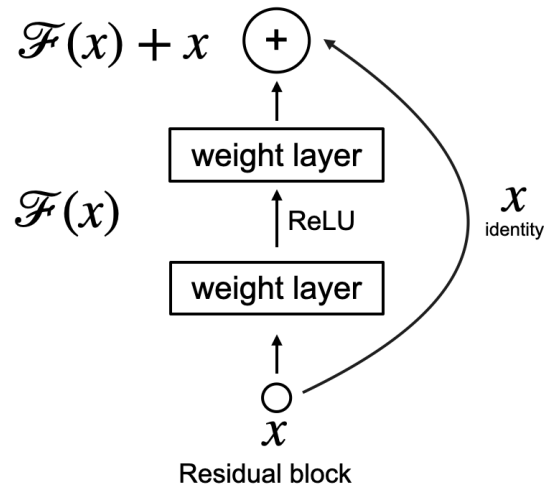
Transformers for Particle physics

* mostly



Replace sums by weighted sums
(**multi-head attn**)

Residual connections





Transformers for Particle physics

* mostly



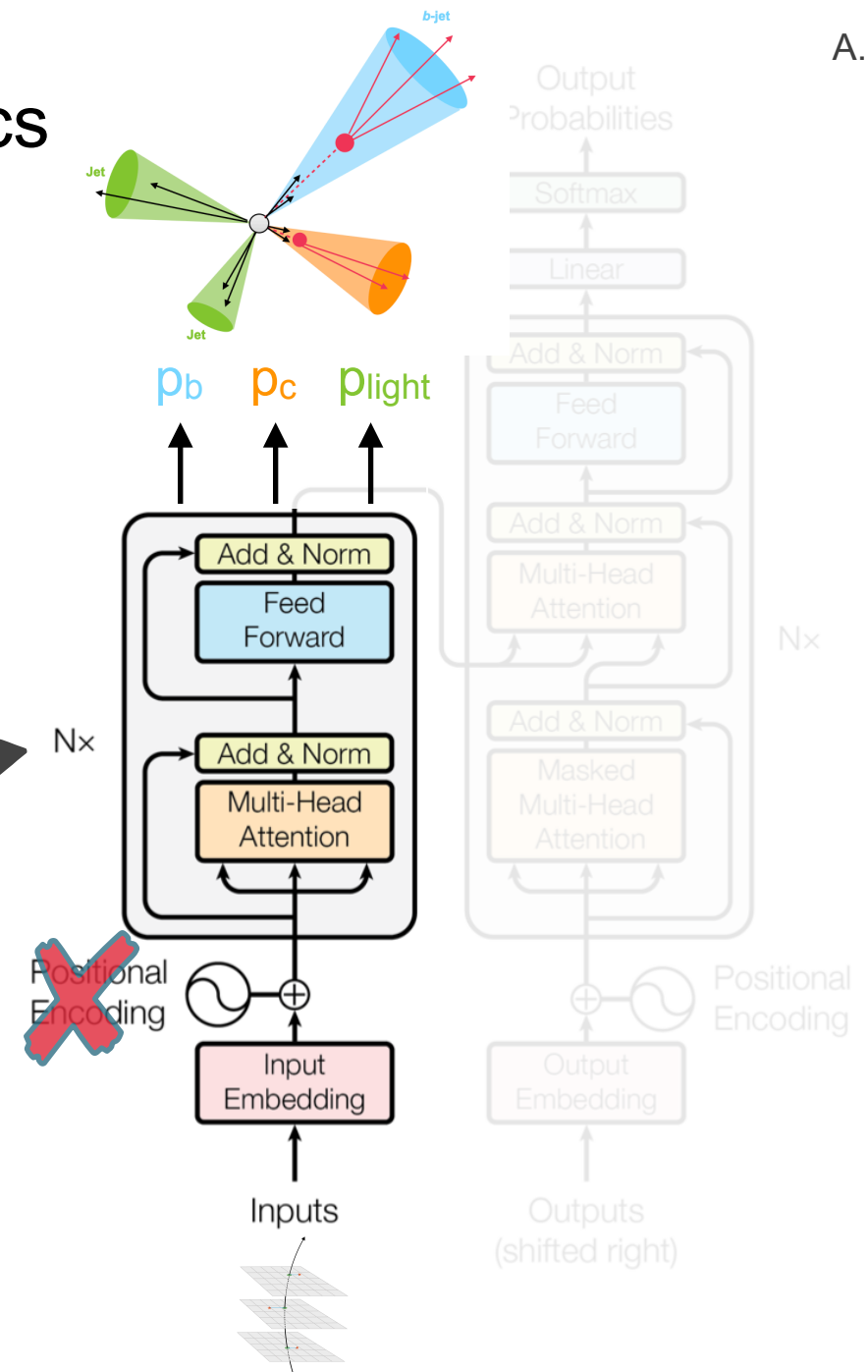
Replace sums by weighted sums
(**multi-head attn**)

Residual connections

Layer normalizaion

The feed forward networks include
dropout ($p=0.1$)

Scale... when you have a good idea,
do it over and over again



Outline

1

How to represent the input

2

Training strategy

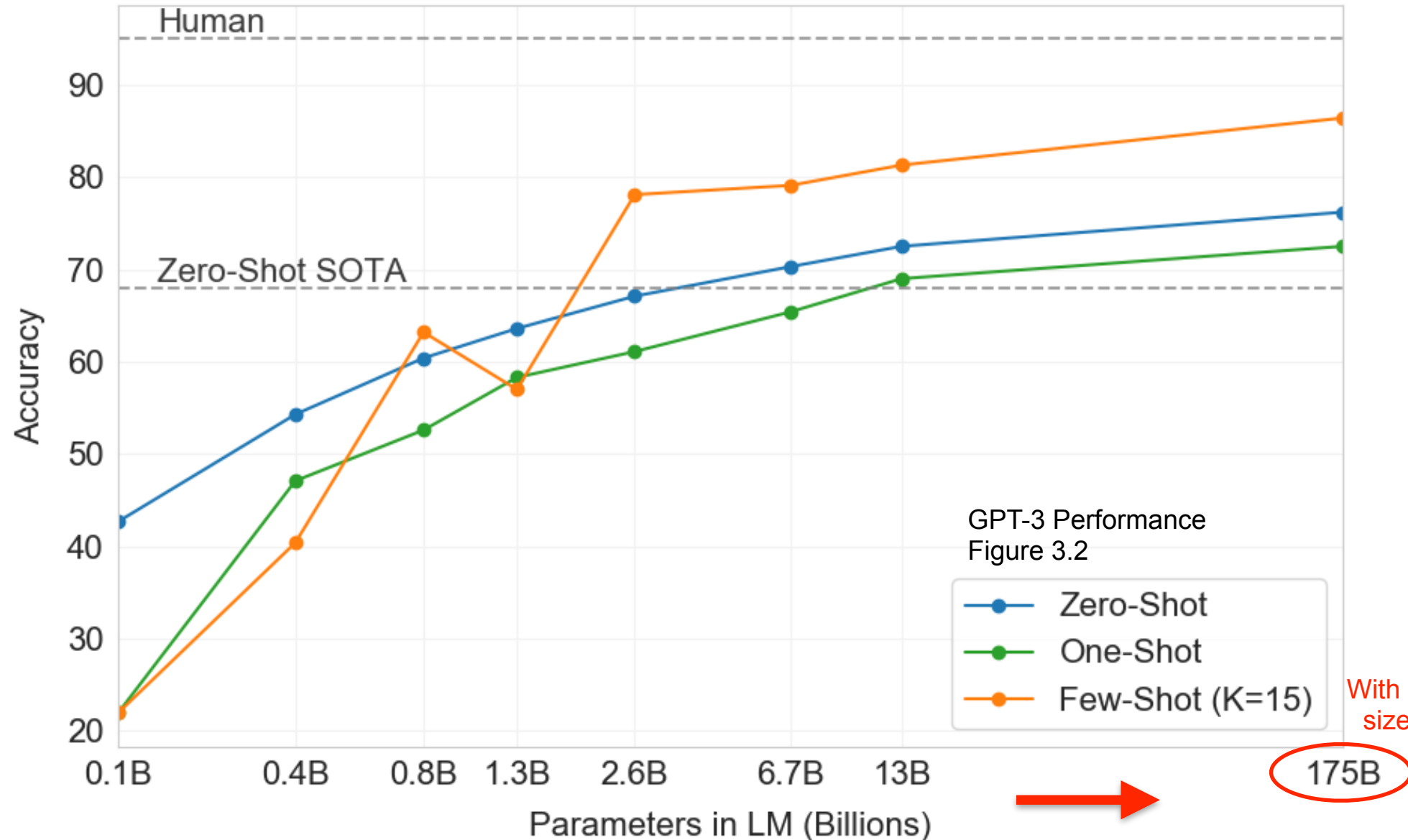
3

Transformers

4

The power of scale

Natural Language Processing

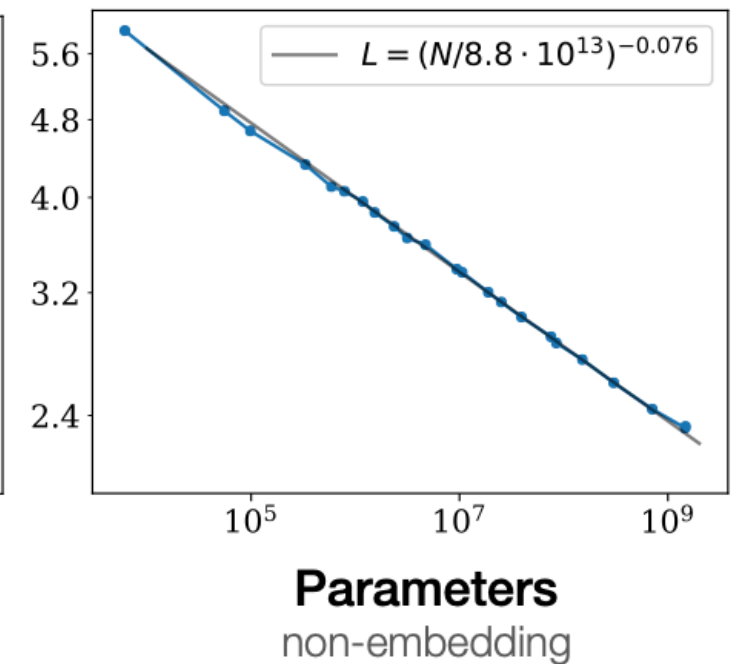
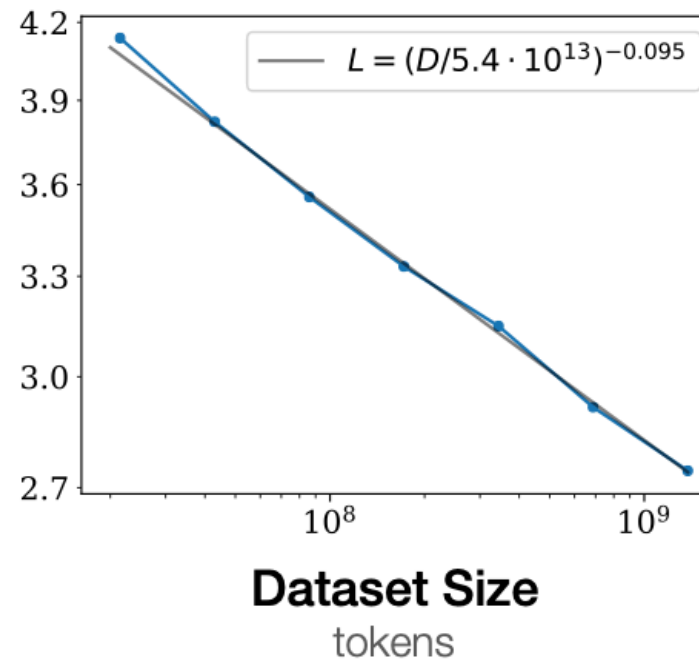
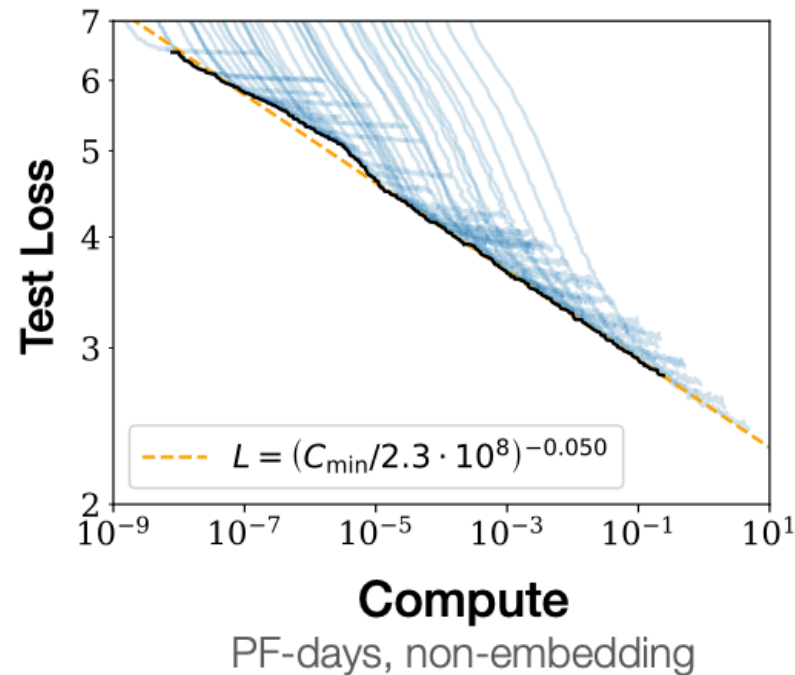


With a training dataset size of 500B words.

Can we *predict* this?

Kaplan: yes!

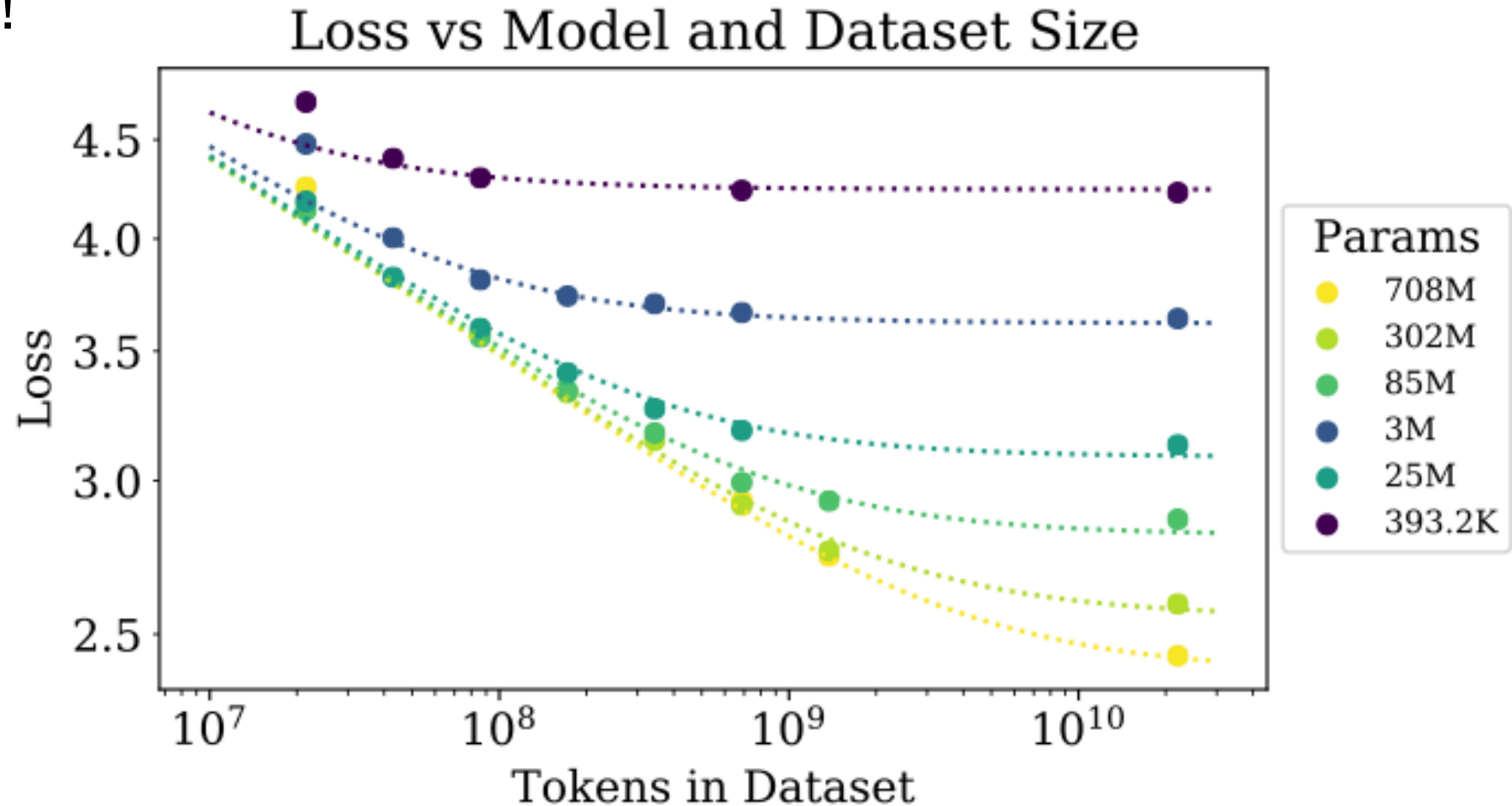
$$\mathcal{L}(N, D) = \mathcal{L}_{\infty} + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$



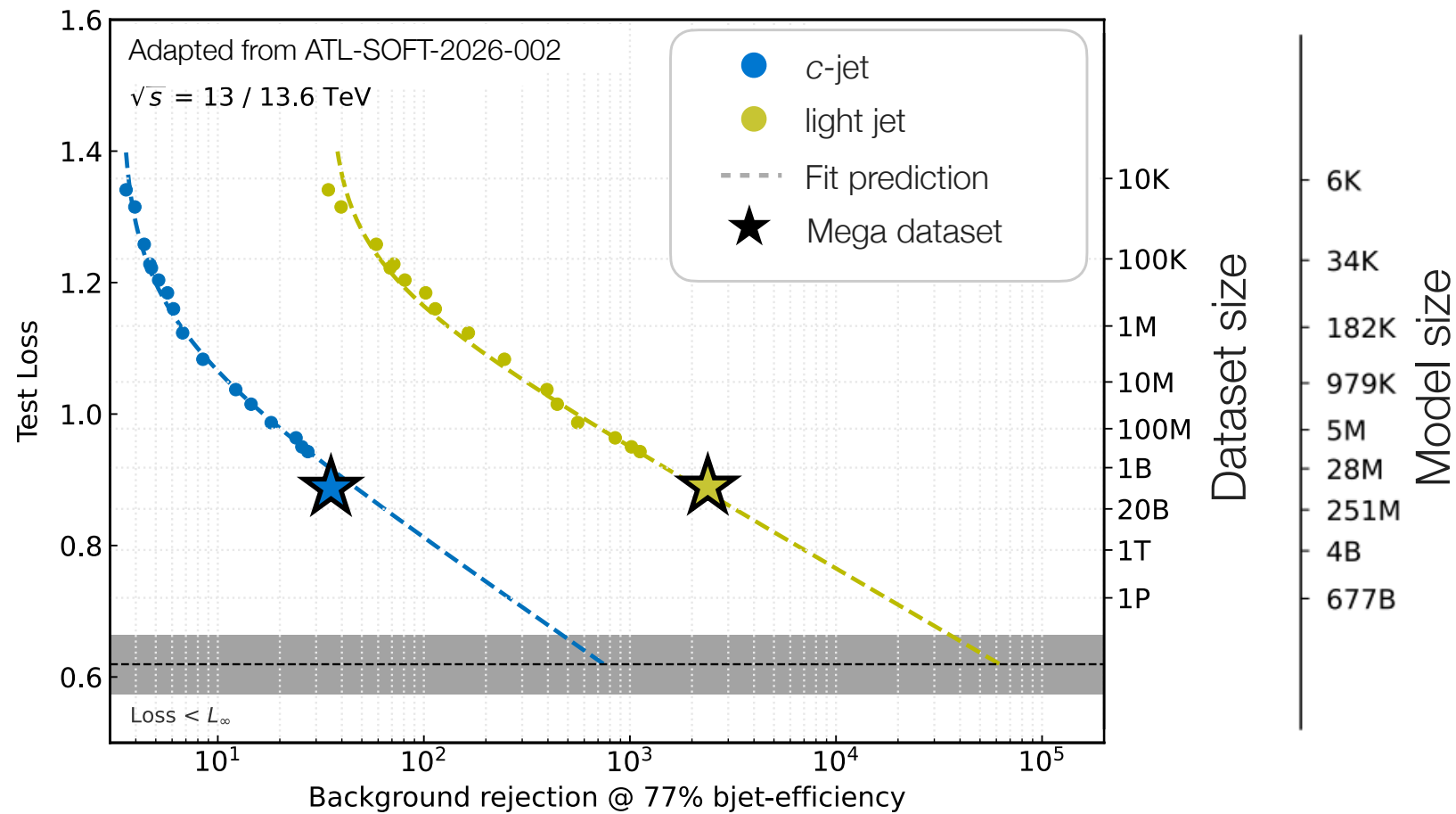
$$C = 6ND$$

Can we *predict* this?

Kaplan: yes!



Also holds for science



GPT-1 (Radford et al., 2018)

- First demonstration of unsupervised pre-training + supervised fine-tuning
- Architecture: 12 layers, 768 dims, 12 heads = 117M parameters
- Trained on BookCorpus (~7,000 books, ~1B tokens)
- Key insight: pre-training on raw text gives transferable representations
- Fine-tuned on specific tasks: QA, classification, entailment

Why it mattered

- Showed transformers could be pre-trained generatively (not just BERT-style)
- Decoder-only architecture with next-token prediction
- Modest size by today's standards but proved the concept
- Directly led to the GPT scaling approach

GPT-2 (Radford et al., 2019)

- "Language models are unsupervised multitask learners"
- Scaled up: 48 layers, 1600 dims, 25 heads = 1.5B parameters
- Trained on WebText: 40GB of text from Reddit outlinks (8M docs)
- Key insight: large LMs can do tasks WITHOUT fine-tuning (zero-shot)
- Generated surprisingly coherent multi-paragraph text
- Initially withheld the largest model due to "misuse concerns"

Technical details

- Context window: 1024 tokens
- Vocabulary: 50,257 BPE tokens
- Pre-norm (LayerNorm before attention/FFN)
- Learned positional embeddings
- No fine-tuning — pure next-token prediction

GPT-3 (Brown et al., 2020)

- 175 billion parameters — 100× larger than GPT-2
- 96 layers, 12,288 dims, 96 heads
- Trained on 300B tokens from Common Crawl + books + Wikipedia
- In-context learning: learn new tasks from a few examples in the prompt
- Zero-shot, one-shot, few-shot performance scales with model size
- Compute: $\sim 3.14 \times 10^{23}$ FLOP ($\sim \$4.6$ M estimated training cost)

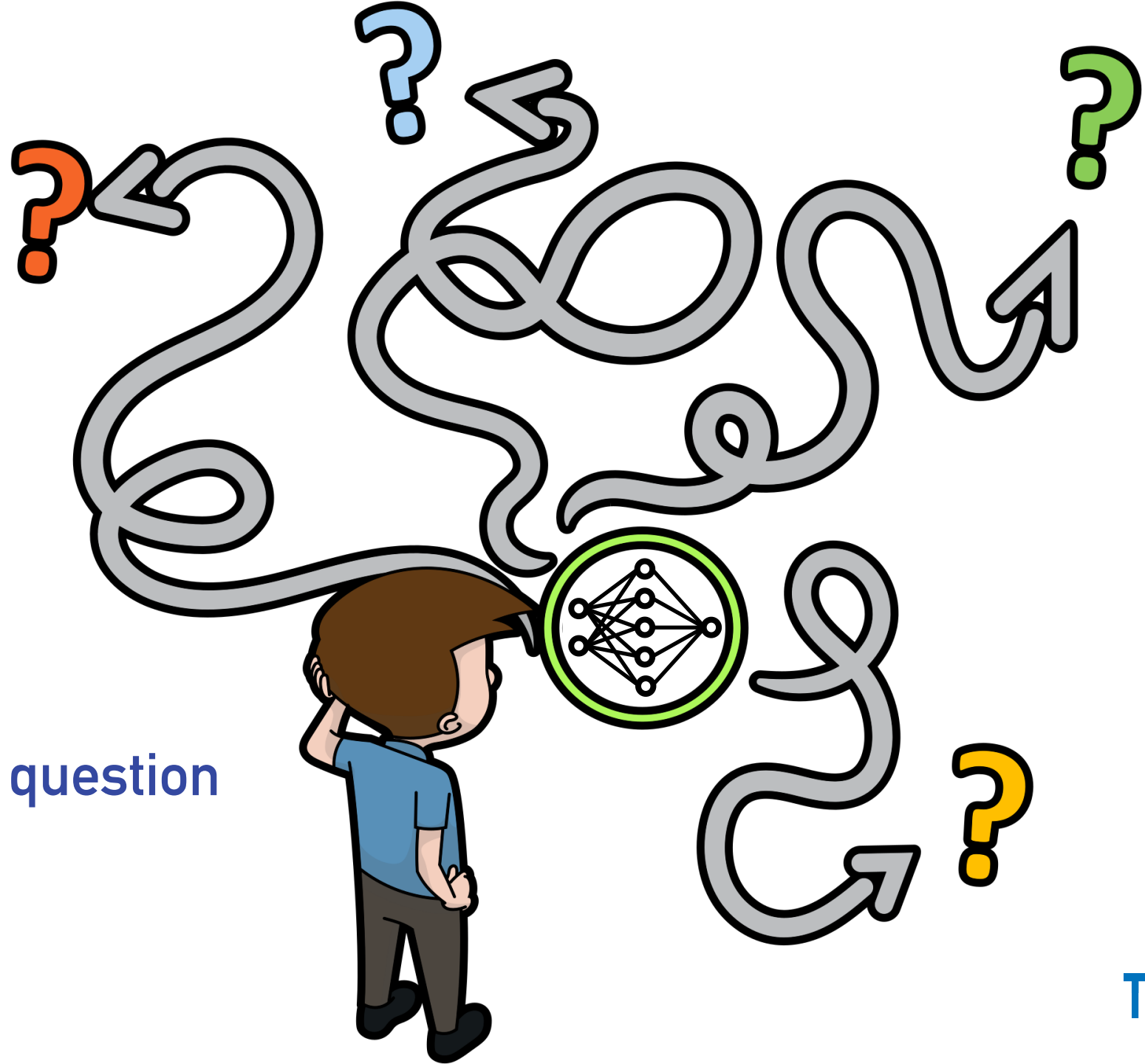
Key capabilities

- Few-shot learning on diverse tasks without gradient updates
- Article writing, translation, code generation
- Arithmetic (limited but improving with scale)
- API access via OpenAI — first commercial LLM API
- Sparked the modern LLM industry

GPT-4 (OpenAI, 2023)

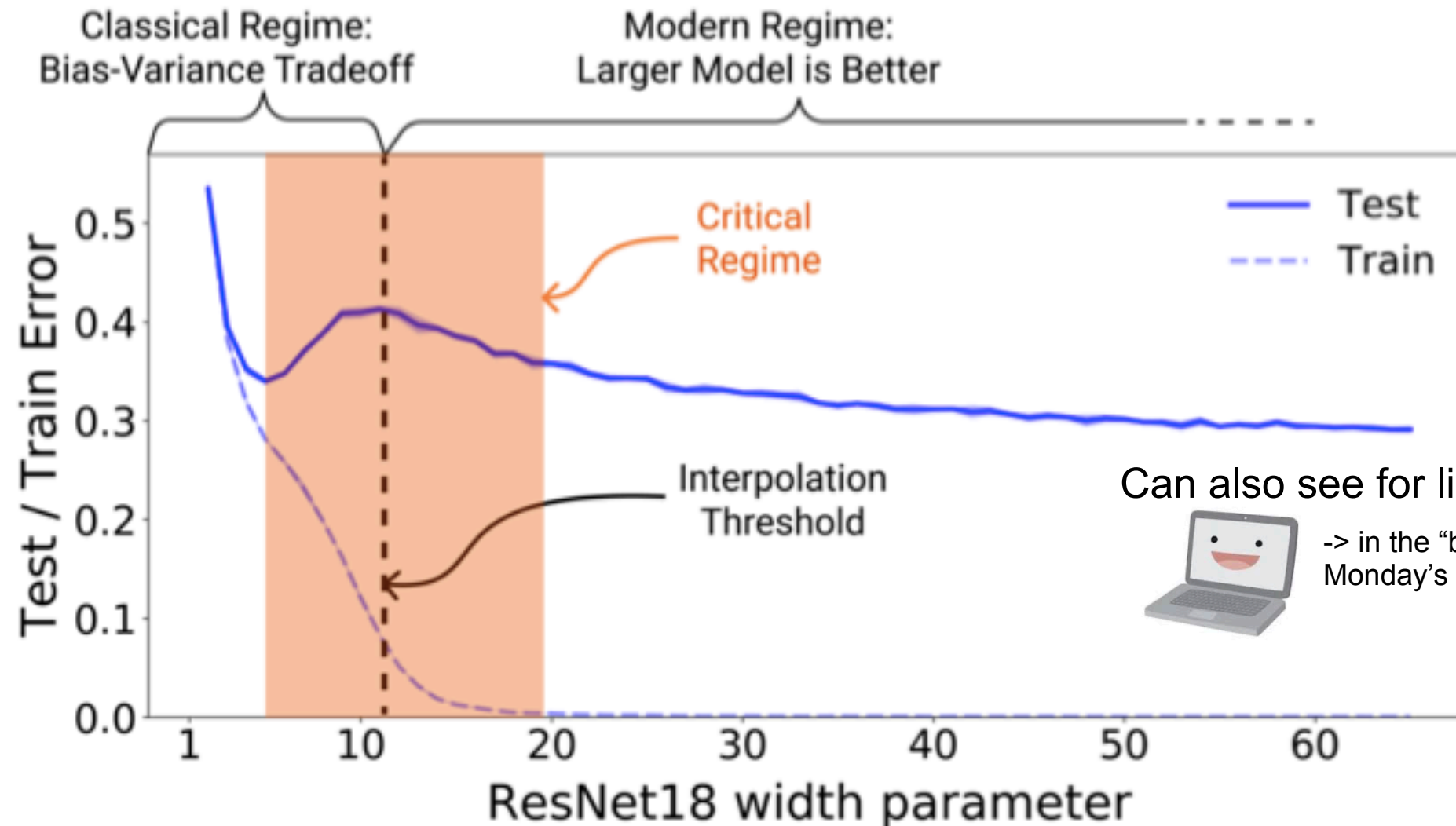
- Estimated ~1.8 trillion parameters (mixture-of-experts architecture, not confirmed)
- Multimodal: accepts both text AND image inputs
- 128k context window (GPT-4-turbo) — ~300 pages of text
- Dramatically better reasoning, instruction following, and safety
- Passes bar exam (90th percentile), SAT, GRE, medical licensing exams
- Trained with RLHF — key to making it safe and helpful
- Training details largely undisclosed (no technical paper with architecture details)

[One of the]
Outstanding deep question
in deep learning.





Double Descent



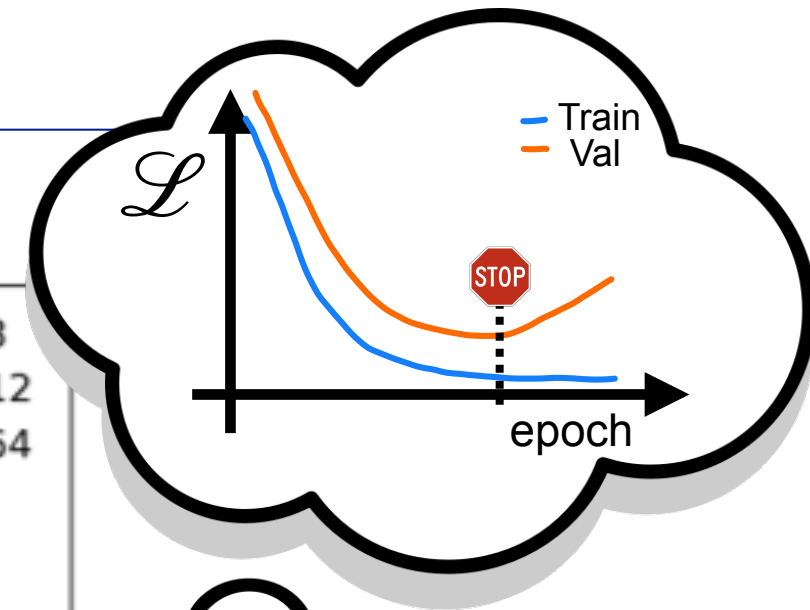
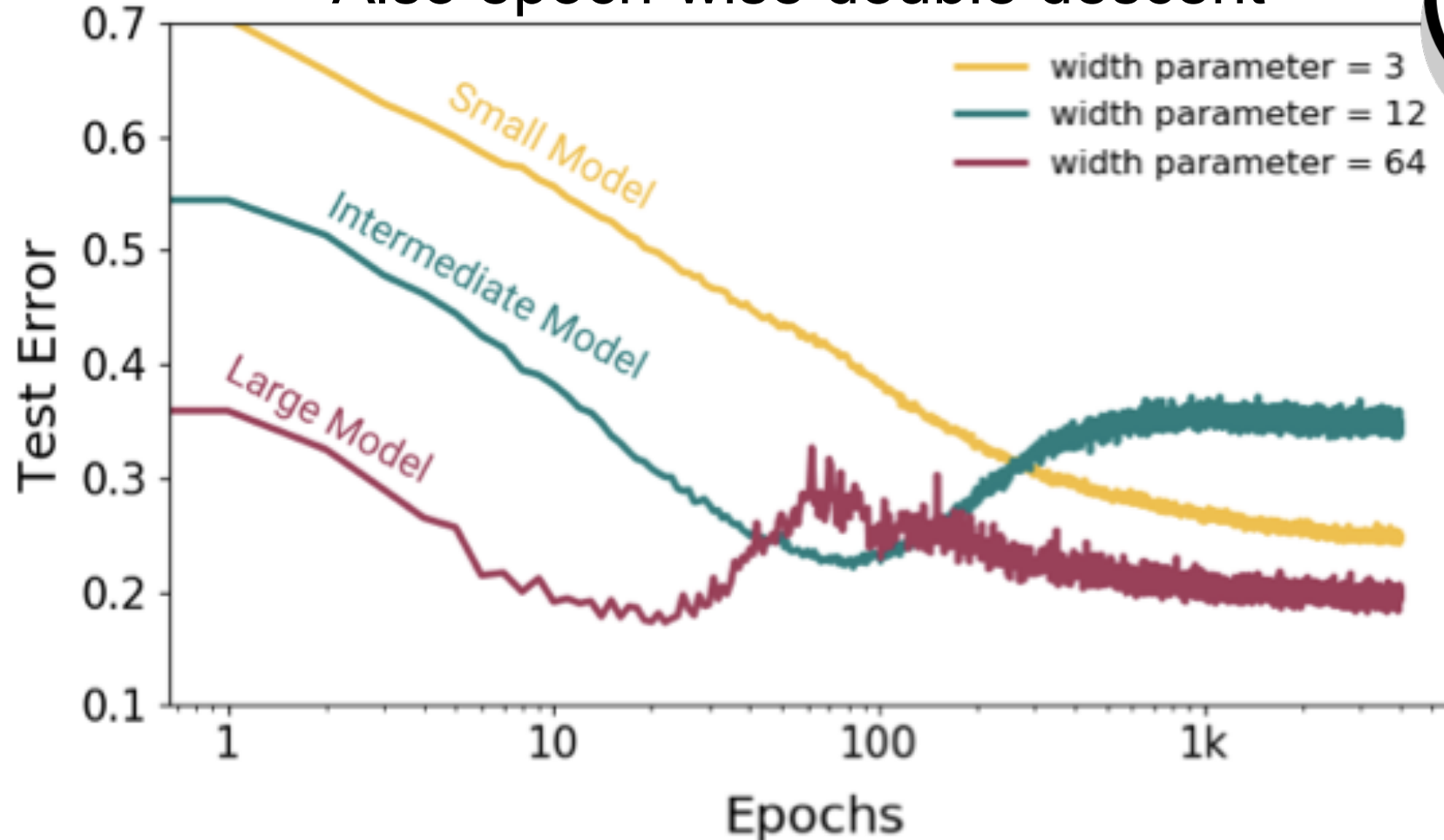
Can also see for linear models!



-> in the "bonus" of Monday's tutorial!

Double Descent

Also epoch-wise double descent



Also from Andrej Karpathy's [blog](#):

6. Squeeze out the juice

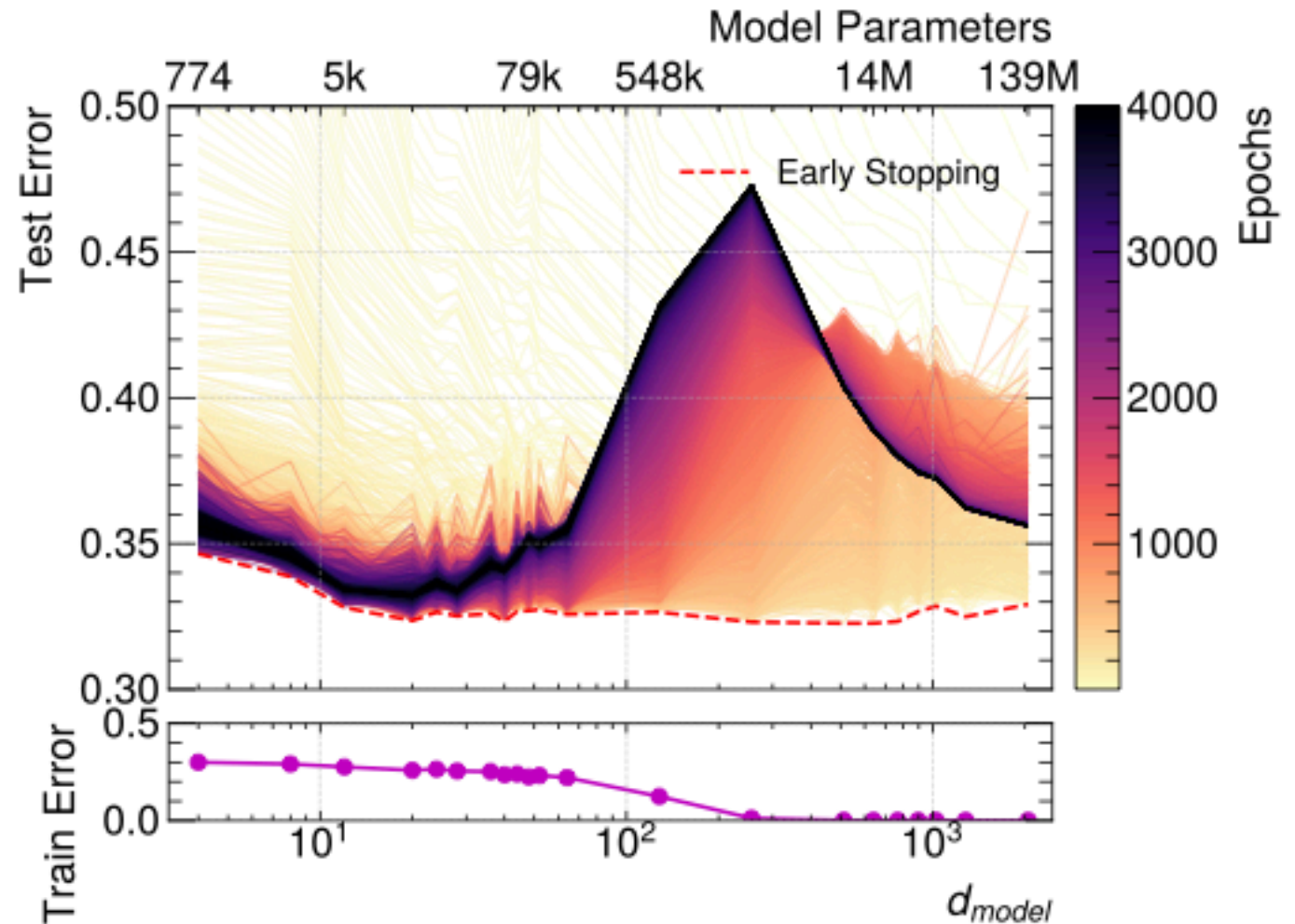
Once you find the best types of architectures and hyper-parameters you can still use a few more tricks to squeeze out the last pieces of juice out of the system:

- **ensembles.** Model ensembles are a pretty much guaranteed way to gain 2% of accuracy on anything. If you can't afford the computation at test time look into distilling your ensemble into a network using [dark knowledge](#).
- **leave it training.** I've often seen people tempted to stop the model training when the validation loss seems to be leveling off. In my experience networks keep training for unintuitively long time. One time I accidentally left a model training during the winter break and when I got back in January it was SOTA ("state of the art").

Double descent in HEP

Training particle regression (jet pt)

- 200k jets (3 orders of magnitude smaller than SOTA datasets)
- order of magnitude more param than previously explored



Lesson: we are far from overparametrized in science!

(Same is true for LLMs 😊)

Architecture summary (week 2) : weight sharing

Alternative architectures designed to *reuse weights* as suited for the input data.

“Don’t relearn what you don’t need”

CNNs

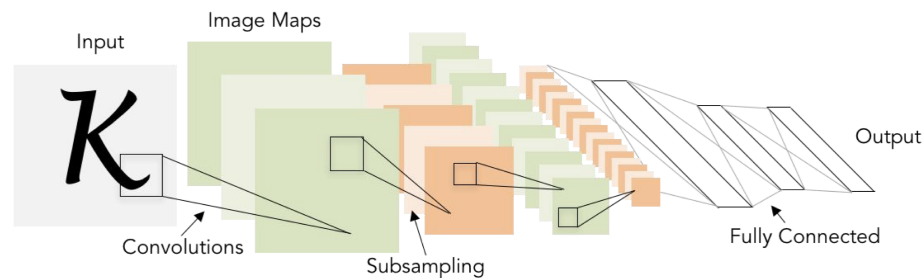
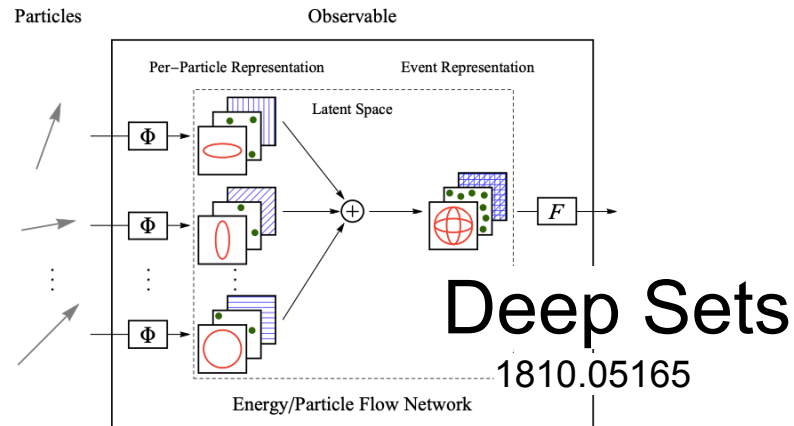
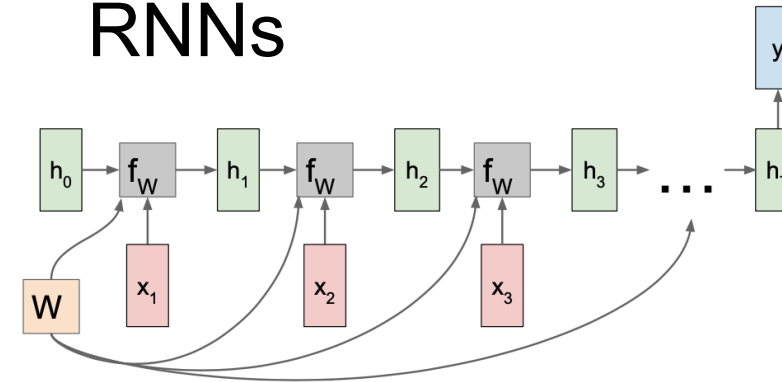


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

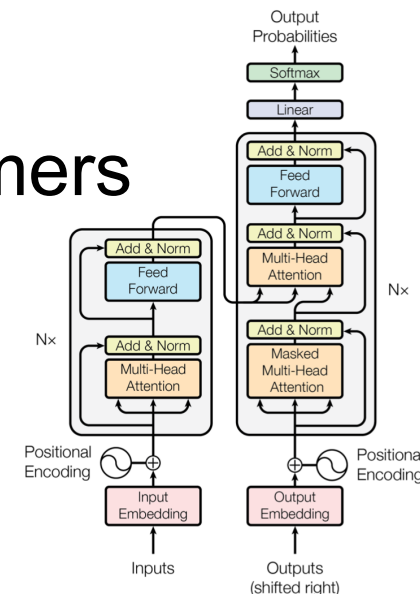


RNNs



Transformers

1706.03762



Exam details

Format: Written exam (questions, space for answering)

Combination of multiple choice, fill in the blank / annotate the plot, and short answer

↪ We are testing concepts, not memorization

Date: 9th April 2026 (time finalizes w/ room booking — we will announce)

Duration: 2 hr

Cheat sheet: 1 page



Bring your student ID!

We will share with you a study guide! (tmr at the latest)

ODSL : What do we do

Origins Data Science Laboratory

Running training events

ML block course (Spring)

Stats block course (Fall)



Informal
chats



Would love to talk! Just an email away: l.heinrich@tum.de,
nicole.hartman@tum.de, baran.hashemi@origins-cluster.de

Building a community



Collaborate on AI
ORIGINS applications



How to stay involved



invite [link](#)

ODSL internal meetings — Wed 16:00

- Chat about your science applications(!)
- Connect on wip results + get feedback

Slack: odsl

Email: odsl-internal@origins-cluster.de

Write an mail to odsl-internal-subscribe@lists.lrz.de. A moderator has to confirm after that.

Seminar series — Fri 14:00

- Share your public science work
- Learn about recent AI for Science applications

Slack: origins-cluster-data-science

Email: odsl@lists.lrz.de

Write an mail to odsl-internal-subscribe@lists.lrz.de. A moderator has to confirm after that.

Backup